



ECOLE DES HAUTES ETUDES D'INGENIERIE – OUJDA

Rapport de Projet de Synthèse

FILIERE : Génie Informatique

G 309

Tourisme local et découverte culturelle

Réalisé par :

Souad HARCHAOUI D

Salah-Eddine ELGHAZI D

Ayoub ARABAT C

Hamza CHDAOUI G

Encadré par :

M. Belkassem ECHCHADLI

Le Jury :

M. Belkassem ECHCHADLI

M. Mohammed Adil MANI

M. Imad MOUHIB

M. Abdelhafid SERGHINI

Année universitaire : 2025- 2026

REMERCIEMENTS

Avant tout développement sur ce travail, il nous apparaît opportun de commencer ce rapport par l'expression de notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à l'élaboration de ce projet de synthèse.

Nous adressons nos plus sincères remerciements à nos encadrants, M. Belkassem Echchadli, M. Mohammed Adil Mani, M. Imad Mouhib et M. Abdelhafid Serghini, pour l'encadrement attentif et bienveillant qu'ils nous ont prodigué tout au long de ce projet. Leurs orientations éclairées, leurs précieux conseils et leur disponibilité constante ont joué un rôle déterminant dans la conduite et la réussite de ce travail.

Nous exprimons également notre reconnaissance à l'ensemble du corps enseignant et administratif de l'École des Hautes Études d'Ingénierie d'Oujda pour la qualité de la formation dispensée et pour avoir mis à notre disposition tous les moyens nécessaires à l'accomplissement de ce projet.

Nous souhaitons enfin remercier nos familles et nos proches pour leur soutien indéfectible, leur patience et leurs encouragements qui ont constitué, tout au long de notre cursus, une source de motivation inestimable. Que tous ceux qui, par leurs remarques et suggestions, ont participé à l'enrichissement de ce travail trouvent ici l'expression de notre sincère gratitude.

TABLE DES MATIÈRES

Contents

REMERCIEMENTS	1
TABLE DES MATIÈRES	2
CHAPITRE I – INTRODUCTION AND DESCRIPTION	1
INTRODUCTION GÉNÉRALE	1
PROBLEMATIQUE	2
SOLUTION	3
DESCRIPTION DU SYSTÈME D'INFORMATION	4
1.1 Objectif Général du Système d'Information	4
1.2 Acteurs Principaux du Système	4
1.3 Fonctionnalités Clés du Système	4
1.3.1 Gestion du Profil Touriste	5
1.3.2 Génération d'Itinéraire	5
1.3.3 Réservation et Paiement	5
1.3.4 Interaction Communautaire et Administration	5
1.4 Structure des Données – Modèle Conceptuel	5
1.5 Flux et Interactions Clés	6
CHAPITRE II – MODÉLISATION	7
1.1 Diagramme de Contexte	7
1.2 Diagramme des Cas d'Utilisation	8
1.3 Descriptions Textuelles des Cas d'Utilisation	9
Titre : S'inscrire	9
Titre : Modifier Profile	10
Titre : activité	11
Titre : Réserver un Restaurant	12
Titre : Réserver un Hôtel	13
Titre : Transport	14
Titre : Visiter une Place Touristique	15
Titre : Paiement	17
Titre : Gérer les Réservations	18
Titre : Analyser les Statistiques	19
Titre : Gérer les Contenus	19
Titre : Mettre à Jour les Disponibilités	20
Titre : Mettre à Jour le Catalogue	21
2. MODÉLISATION TECHNIQUE	23
2.1 Diagramme de Classes UML	23
2.2 Modèle Conceptuel de Données (MCD)	24

2.3	Modèle Logique de Données (MLD).....	25
2.4	Script de Création de la Base de Données	26
2.5	Concepts de la Programmation Orientée Objet	29
2.6	Architecture Technique Globale	29
2.6.1	Vue d'ensemble des Trois Composants	30
2.6.2	Patron de Conception MVC — Appliqué dans les 3 Technologies	30
2.6.3	Justification du Choix Technologique.....	30
3	GESTION DE PROJET	32
3.1	Méthodologie de Développement.....	32
3.1.1	Le Manifeste Agile et ses Valeurs Fondatrices	32
3.1.2	Le Cycle en Y — Double Branche de Conception	32
3.2	Organisation Scrum de l'Équipe	33
3.3	Product Backlog et Sprints de Développement	33
3.4	Cycle de Vie du Projet — Six Phases Clés.....	33
3.5	Planification — Diagramme de Gantt	34
3.6	Importance du Travail en Équipe	35
3.1	Site Web – Elyora — Interface - Touriste	36
3.1.1	Page d'accueil du site Elyora	36
3.1.2	Page d'inscription (Register)	37
3.1.3	Page de connexion (Login)	38
3.1.4	Page des Préférences de Voyage	39
3.1.5	Page de l'Itinéraire Personnalisé	40
3.1.6	Page d'Exploration des Lieux (Destinations)	41
3.1.7	Page de Gestion des Réservations (My Bookings)	42
3.1.8	Page de Modification du Profil (Edit Account)	43
3.2	Console Java — Admin - Rapports	44
3.2.1	Menu Principal	44
3.2.2	Statistiques & Rapports.....	45
3.2.3	Rapport Complet	46
3.2.4	Compter par statut	47
3.2.5	Voir toutes les réservations	48
3.2.6	Voir toutes les réservations	49
3.3	Console C++ — Admin – Operations	50
3.3.1	Interface du panneau d'administration	50
3.3.2	Affichage de toutes les chambres	51
3.3.3	Ajout d'une nouvelle chambre.....	52
3.3.4	Suppression d'une chambre existante.....	53
	CONCLUSION GÉNÉRALE	54
	RÉFÉRENCES ET BIBLIOGRAPHIE	55
	Ouvrages et Références Académiques	55
	Ressources Techniques	55

CHAPITRE I — INTRODUCTION AND DESCRIPTION

INTRODUCTION GÉNÉRALE

Dans un contexte mondial où le tourisme tend vers une massification croissante et une standardisation des expériences, les voyageurs contemporains aspirent à des expériences plus profondes, plus humaines et plus significatives. Il ne s'agit plus uniquement de visiter un lieu, mais de le comprendre dans toute sa complexité, de le ressentir dans son authenticité et d'y inscrire une empreinte positive et durable.

C'est précisément dans ce contexte émergent d'un tourisme plus conscient, éthique et personnalisé qu'est né le projet Elyora — une application mobile conçue non pas pour guider passivement les touristes, mais pour les accompagner dans une découverte véritablement authentique du territoire, de ses habitants et de ses richesses culturelles souvent méconnues.

Elyora se veut bien plus qu'un simple outil numérique : c'est un pont entre tradition et innovation, entre curiosité individuelle et patrimoine collectif. En valorisant les savoir-faire locaux, les initiatives culturelles et les lieux méconnus, l'application propose une alternative durable à l'offre touristique conventionnelle, fondée sur la qualité de l'expérience plutôt que sur la quantité de prestations, sur la rencontre humaine plutôt que sur la consommation touristique.

Le présent rapport expose la vision fondatrice, les enjeux technologiques et les fonctionnalités clés d'Elyora, en réponse à un besoin croissant de tourisme personnalisé, éthique et profondément connecté à l'âme des territoires. Il s'articule autour de plusieurs chapitres traitant successivement de la description du système d'information, de la modélisation fonctionnelle et technique, des méthodes de gestion de projet adoptées, ainsi que des principaux artefacts de développement produits par l'équipe.

Elyora — Passerelle entre technologie et tradition, voyageur et habitant, innovation numérique et richesses culturelles locales.

PROBLEMATIQUE

Le tourisme moderne souffre aujourd'hui d'un manque d'authenticité et de personnalisation. La plupart des touristes suivent des itinéraires standards proposés par les agences ou les applications classiques, ce qui limite la découverte réelle de la culture locale. Les visiteurs fréquentent souvent les mêmes lieux touristiques sans avoir de véritable contact avec les traditions, les habitants ou les spécialités régionales.

Pourtant, les attentes des voyageurs ont évolué. Ils recherchent désormais des expériences plus immersives et adaptées à leurs centres d'intérêt comme la gastronomie, le patrimoine, la nature ou les arts. Cependant, les outils numériques existants offrent rarement des recommandations personnalisées permettant une véritable immersion culturelle.

Ainsi, il devient nécessaire de développer un outil numérique simple et interactif capable de proposer des expériences touristiques personnalisées tout en valorisant la culture locale et les richesses authentiques des régions.

SOLUTION

Elyora est une application intelligente conçue pour offrir une expérience touristique plus authentique et personnalisée. Elle répond au problème de la standardisation du tourisme en proposant des itinéraires adaptés aux préférences de chaque utilisateur, selon ses centres d'intérêt, son budget et la durée de son séjour.

Grâce à un profil personnalisé, l'application recommande des activités liées à la gastronomie, au patrimoine, à la nature, aux arts ou aux traditions locales. Une carte interactive permet aux visiteurs d'explorer facilement les lieux touristiques, les restaurants, les hôtels, les activités et les services locaux disponibles dans la région.

Elyora intègre également des fiches culturelles enrichies afin de mieux faire découvrir l'histoire, les traditions et l'identité des destinations visitées. En plus de cela, l'utilisateur peut réserver directement différents services comme les guides locaux, les hébergements ou les activités touristiques depuis l'application.

Enfin, un espace communautaire favorise l'échange d'expériences et de recommandations entre voyageurs et habitants, renforçant ainsi l'immersion culturelle et la découverte authentique de la région.

DESCRIPTION DU SYSTÈME D'INFORMATION

1.1 Objectif Général du Système d'Information

Le système d'information Elyora est conçu pour gérer l'ensemble des interactions entre les utilisateurs — touristes et administrateurs — et les services numériques proposés par l'application. Il vise à automatiser la création d'expériences touristiques personnalisées, la gestion des réservations, le traitement sécurisé des paiements, ainsi que la mise en relation directe avec les acteurs économiques locaux. Ce tout en garantissant une expérience utilisateur fluide, cohérente et culturellement enrichissante.

1.2 Acteurs Principaux du Système

Le système d'information d'Elyora implique deux catégories d'acteurs principaux, dont les rôles et responsabilités sont clairement définis :

- **Touriste** : Le Touriste constitue l'utilisateur final de l'application. Il interagit avec le système pour créer et personnaliser son profil, générer des itinéraires adaptés à ses préférences, réserver des activités et services locaux, procéder aux paiements correspondants, et partager ses expériences avec la communauté.
- **Administrateur** : L'Administrateur est responsable de la gestion globale du système. Il analyse les statistiques d'utilisation, administre le catalogue de services disponibles, gère les disponibilités des ressources et supervise les réservations entrantes.

1.3 Fonctionnalités Clés du Système

Le système d'information permet de réaliser les cas d'utilisation suivants, organisés selon quatre grandes catégories fonctionnelles :

1.3.1 Gestion du Profil Touriste

- Inscription : création d'un compte utilisateur avec saisie des informations personnelles.
- Modification du profil : mise à jour des préférences (centres d'intérêt, budget, durée de séjour, mode de transport, langues).

1.3.2 Génération d'Itinéraire

- Visite de sites touristiques : consultation des lieux culturels et intégration dans un itinéraire personnalisé.
- Guide touristique : accès aux fiches culturelles enrichies incluant historique, photographies et contenus vidéo.
- L'itinéraire est généré automatiquement lors de la sélection d'activités et de lieux, en optimisant le temps de parcours et la distance géographique.

1.3.3 Réservation et Paiement

- Réservation de restaurant, d'hôtel, de transport et d'activités culturelles.
- Paiement sécurisé par carte bancaire, en espèces (géré par le prestataire local) ou via une passerelle de paiement en ligne. Le paiement est obligatoirement requis pour valider toute réservation.

1.3.4 Interaction Communautaire et Administration

- Partage d'expériences : publication d'avis et de commentaires sur les lieux et activités visités.
- Affichage des activités : consultation du catalogue des activités proposées par les administrateurs.
- Gestion des réservations et mise à jour du catalogue par les prestataires locaux.
- Analyse statistique : suivi des performances du système par l'administrateur central.
- Gestion des contenus : validation et modération des contenus publiés.

1.4 Structure des Données — Modèle Conceptuel

Le système d'information repose sur une structure de données organisée autour des entités principales suivantes :

- ▶ Touriste : contient les données personnelles et les méthodes de gestion du profil et des réservations.
- ▶ Profil (Séjour) : stocke les préférences du touriste, notamment le budget, les centres d'intérêt et le mode de transport préféré. Chaque touriste dispose d'exactly un profil.
- ▶ Itinéraire : représente un parcours personnalisé avec un temps total et une distance calculée, composé de points d'intérêt (lieux touristiques, activités, transports).
- ▶ LieuTouristique : centralise les informations culturelles enrichies (nom, description, histoire, photos, vidéos, horaires, accessibilité). Un lieu peut appartenir à plusieurs itinéraires.
- ▶ Activité : décrit une activité culturelle ou de loisir (nom, description, prix, durée, lieu associé). Elle peut être réservée et intégrée dans un itinéraire.
- ▶ Restaurant / Hôtel : entités prestataires disposant de leurs propres catalogues (plats, chambres) et disponibilités, liées aux réservations.
- ▶ Réservation : entité centrale liant un touriste, un service et un paiement. Son statut évolue selon le cycle : En Attente → Confirmé → Payé / Annulé.
- ▶ Paiement : classe abstraite déclinée en trois sous-classes (PaiementCarte, PaiementCash, PaiementEnLigne). Chaque réservation est associée à un paiement optionnel.
- ▶ Administrateur (Admin/Admine) : dispose des méthodes pour analyser les données et gérer les contenus, les réservations et les mises à jour du catalogue.

1.5 Flux et Interactions Clés

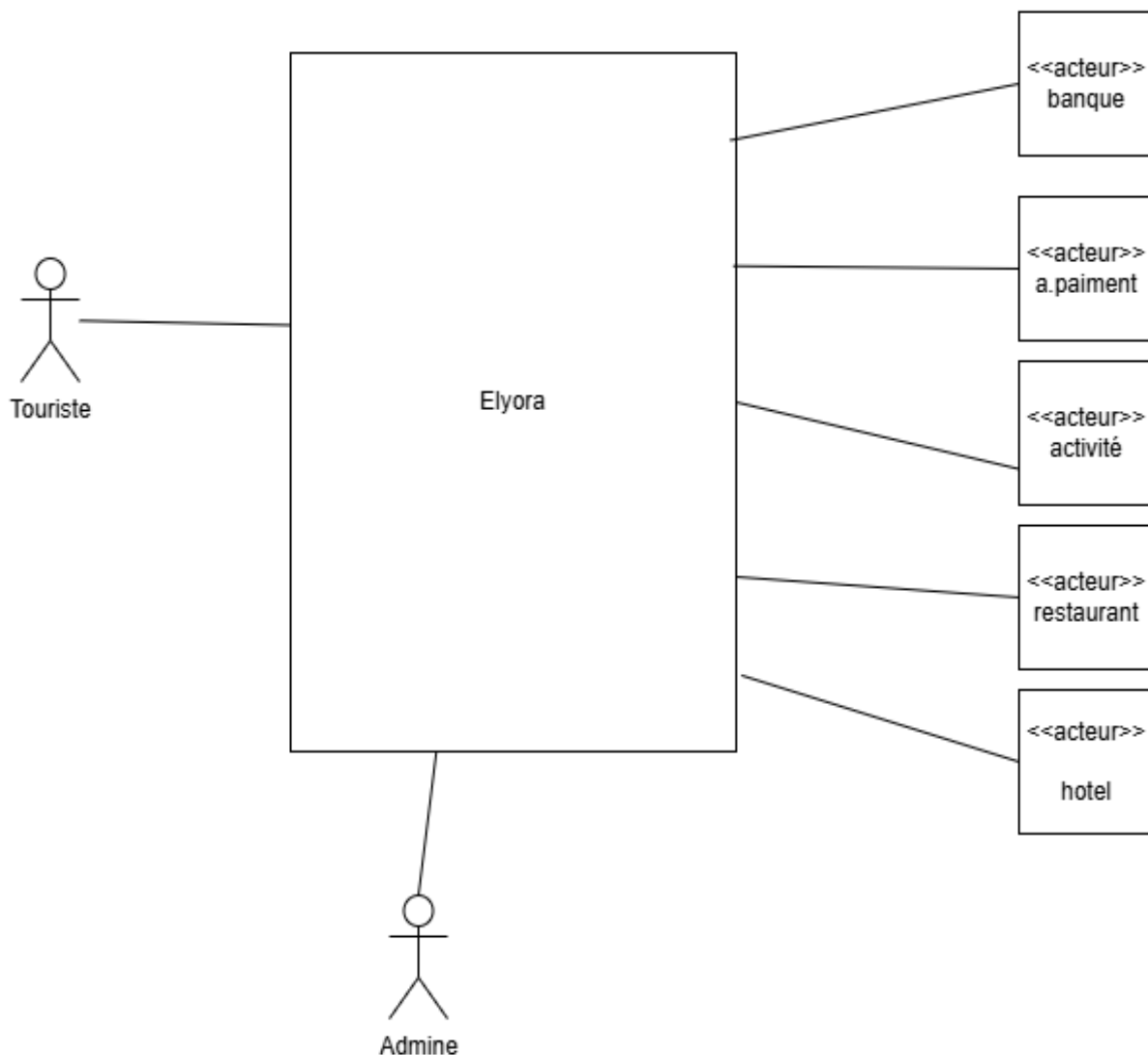
Les principaux flux d'interactions au sein du système d'information Elyora s'articulent comme suit :

- 1. Création d'expérience :** Le touriste crée son profil, le système génère un itinéraire personnalisé, le touriste ajoute des activités et des lieux, effectue ses réservations et son paiement, visite les sites, puis partage son expérience.
- 2. Gestion des réservations :** L'administrateur reçoit une réservation, la confirme ou l'annule, et met à jour ses disponibilités en conséquence.
- 3. Traitement du paiement :** La réservation déclenche un processus de paiement. Selon le mode sélectionné, le système communique avec l'établissement bancaire (carte/en ligne) ou inscrit la réservation comme payable en espèces.
- 4. Administration :** L'administrateur central surveille les indicateurs statistiques et modère les contenus afin de garantir la qualité et la fiabilité de l'ensemble du système.

CHAPITRE II — MODÉLISATION

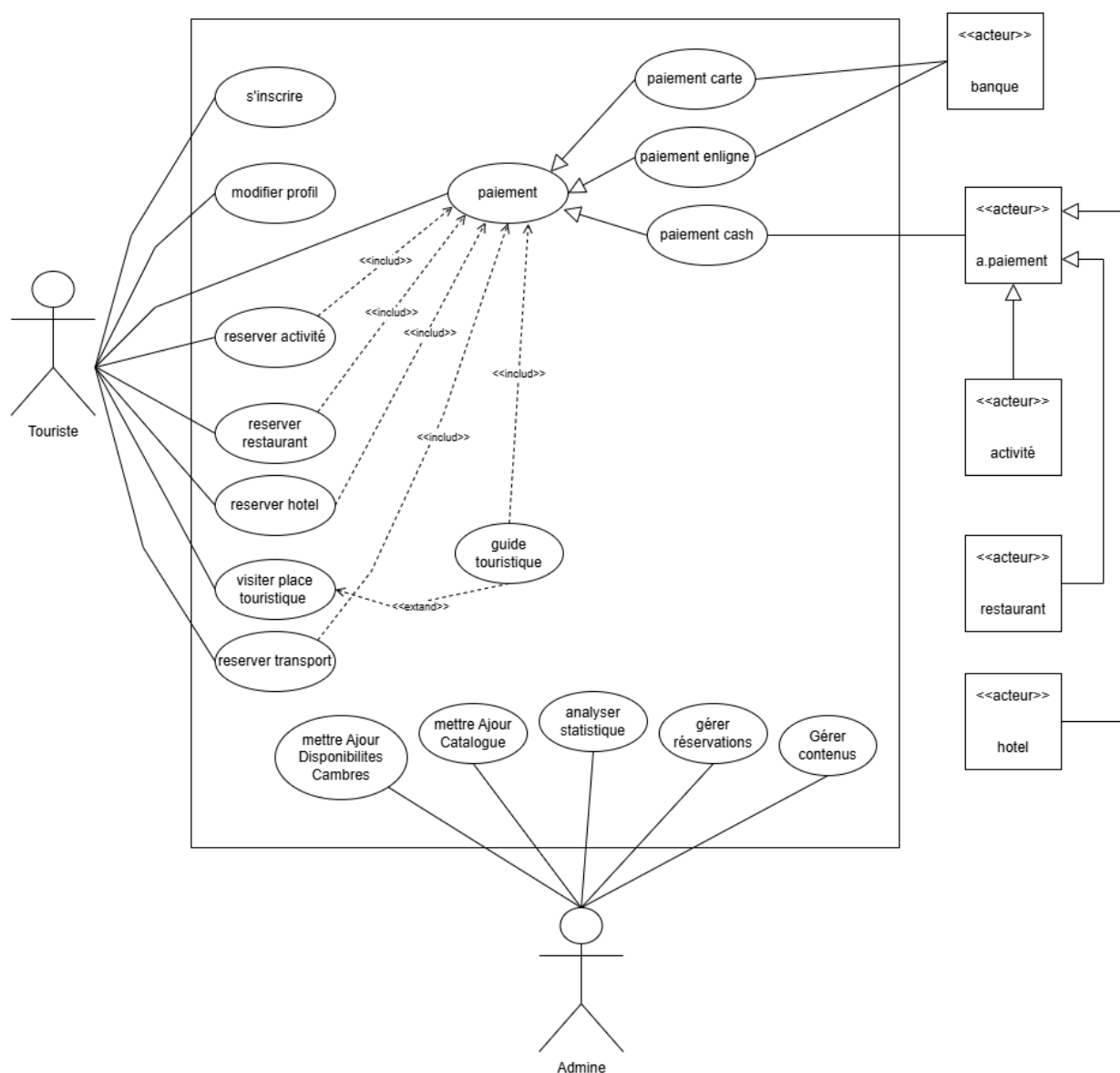
1.1 Diagramme de Contexte

Le diagramme de contexte positionne le système Elyora dans son environnement fonctionnel global et identifie les interactions avec l'ensemble des acteurs externes. Il délimite le périmètre du système en distinguant clairement les flux d'information entrants et sortants, permettant ainsi une vision synthétique des interfaces entre le système et son écosystème.



1.2 Diagramme des Cas d'Utilisation

Le diagramme des cas d'utilisation d'Elyora recense l'ensemble des interactions fonctionnelles entre les acteurs principaux (Touriste et Administrateur) et le système. Quatorze cas d'utilisation ont été identifiés, incluant des relations de type `<<include>>` (dépendances obligatoires telles que le paiement pour toute réservation) et `<<extend>>` (extensions optionnelles telles que la réservation d'un guide touristique).



1.3 Descriptions Textuelles des Cas d'Utilisation

Les sections suivantes présentent les descriptions textuelles détaillées de chacun des cas d'utilisation identifiés dans le diagramme. Chaque cas est documenté selon la structure normalisée incluant l'objectif, les acteurs concernés, les pré-conditions et post-conditions, ainsi que les scénarios nominaux et alternatifs.

Titre : S'inscrire

Résumé :

Ce cas d'utilisation décrit le processus permettant à un nouveau touriste de créer un compte sur l'application Elyora. Le touriste accède à l'option « S'inscrire », puis remplit un formulaire avec ses informations personnelles (email, mot de passe, nom, prénom et téléphone). Le système vérifie la validité des données saisies, notamment le format de l'email et la sécurité du mot de passe. Si les informations sont correctes, un compte utilisateur est créé avec un profil initial, puis l'utilisateur est redirigé vers son profil avec un message de confirmation indiquant que le compte a été créé avec succès.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur n'a pas encore de compte sur Elyora.

Scénario nominal :

1. Le touriste lance l'application et sélectionne l'option « S'inscrire ».
2. Le système affiche le formulaire d'inscription.
3. Le touriste renseigne ses informations (adresse électronique, mot de passe, nom, prénom, numéro de téléphone).

4. Le système valide les informations saisies (format de l'adresse électronique, complexité du mot de passe).
5. Le système crée le compte et redirige l'utilisateur vers son profil.
6. Le système affiche un message de confirmation : « Votre compte a été créé avec succès ».

Enchaînements alternatifs :

A1 : Informations invalides (étape 4)

5. Le système affiche un message d'erreur précis (ex : "Adresse email invalide" ou "Le mot de passe doit contenir au moins 8 caractères").
6. Le touriste corrige les informations et réessaie.

A2 : Email déjà utilisé (étape 4)

5. Le système affiche un message : "Un compte existe déjà avec cette adresse email.
6. Veuillez-vous connecter ou utiliser une autre adresse."
7. Le touriste peut choisir de se connecter ou de revenir au formulaire avec une nouvelle adresse.

Postconditions :

- Un compte utilisateur est créé avec un profil initial.

Titre : Modifier Profile**Résumé :**

Le touriste connecté peut modifier ses informations personnelles et préférences depuis la section « Mon Profil ». Après avoir mis à jour les champs souhaités et enregistré les modifications, le système valide et sauvegarde les données puis affiche un message de confirmation. En cas d'erreur, le système indique les champs invalides ou demande de réessayer plus tard.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté à son compte.

Scénario nominal :

1. Le touriste navigue jusqu'à la section "Mon Profil".
2. Le système affiche le profil actuel de l'utilisateur.
3. Le touriste clique sur "Modifier".
4. Le système affiche un formulaire pré-rempli avec les données actuelles.
5. Le touriste modifie les champs souhaités (centres d'intérêt, budget, durée de séjour, mode de transport, etc.).
6. Le touriste clique sur "Enregistrer".
7. Le système valide les modifications.
8. Le système met à jour le profil et affiche un message de confirmation : "Vos informations ont été mises à jour."

Enchaînements alternatifs :

A1 : Données invalides (étape 7)

8. Le système affiche un message d'erreur indiquant le champ invalide.
9. Le touriste corrige les données et réessaie.

A2 : Échec de sauvegarde (étape 7)

8. Le système affiche un message : "Une erreur est survenue. Veuillez réessayer plus tard".
9. Le touriste peut tenter de sauvegarder à nouveau ou annuler les modifications.

Postconditions :

- Le profil de l'utilisateur est mis à jour avec les nouvelles informations.

Titre : activité**Résumé :**

Le touriste connecté peut découvrir des activités culturelles, artisanales ou de loisir adaptées à ses centres d'intérêt. Il peut consulter les détails d'une activité puis choisir de passer à la réservation.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté et a un profil (même partiellement rempli).

Scénario nominal :

1. Le touriste navigue vers la section "Activités".
2. Le système affiche une liste d'activités personnalisées (atelier artisanal, visite guidée, spectacle local, etc.) basées sur les centres d'intérêt du profil.
3. Le touriste parcourt la liste ou utilise les filtres (type, prix, distance).
4. Le touriste clique sur une activité pour voir sa fiche culturelle enrichie (description, anecdotes, photos, vidéos).
5. Le touriste décide soit de fermer la fiche, soit de passer à la réservation (ce qui déclenche le cas d'utilisation `paiement`).

Enchaînements alternatifs :

A1 : Aucune activité disponible (étape 2)

3. Le système affiche un message : "Aucune activité correspondant à vos critères n'a été trouvée. Essayez d'élargir votre recherche".

4. trouvée. Essayez d'élargir votre recherche."

Le touriste ajuste ses filtres ou explore d'autres catégories

A2 : Erreur de chargement de la fiche (étape 4)

5. Le système affiche un message : "Impossible de charger les détails de cette activité. Veuillez réessayer plus tard."

6. Le touriste peut rafraîchir la page ou choisir une autre activité.

Postconditions :

- L'utilisateur a consulté les activités disponibles et peut en sélectionner une pour plus de détails ou une réservation.

Titre : Réserver un Restaurant**Résumé :**

Le touriste connecté peut réserver une table dans un restaurant local selon ses préférences. Après avoir choisi le restaurant, la date et l'heure, le système vérifie la disponibilité puis confirme la réservation après le paiement.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté.

Scénario nominal :

1. Le touriste accède à la section « Restaurants ».
2. Le système affiche une liste de restaurants adaptés aux préférences du profil.
3. Le touriste consulte la fiche d'un restaurant.
4. Le touriste choisit la date, l'heure et le nombre de couverts.
5. Le système vérifie la disponibilité.
6. Le système lance le processus de paiement.
7. Le système confirme la réservation et envoie une notification.
8. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Aucune disponibilité(étape 5)

6. Le système informe qu'aucune table n'est disponible au créneau choisi.
7. Le touriste modifie ses choix ou sélectionne un autre restaurant.

A2 : Échec du paiement(étape 6)

7. Le système affiche un message d'erreur de paiement.
8. Le touriste peut réessayer ou annuler la réservation.

Postconditions :

- Une réservation est confirmée et enregistrée dans le système.

Titre : Réserver un Hôtel**Résumé :**

Le touriste connecté peut réserver une chambre d'hôtel selon ses préférences. Après avoir choisi l'hôtel, les dates et le type de chambre, le système vérifie la disponibilité puis confirme la réservation après le paiement.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté.

Scénario nominal :

1. Le touriste accède à la section « Hôtels ».
2. Le système affiche une liste d'hébergements adaptés aux préférences du profil.
3. Le touriste consulte la fiche d'un hôtel.
4. Le touriste choisit les dates de séjour et le type de chambre.
5. Le système vérifie la disponibilité.
6. Le système lance le processus de paiement.
7. Le système confirme la réservation et envoie une notification.
8. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Aucune disponibilité (étape 5)

6. Le système informe qu'aucune chambre n'est disponible pour les dates choisies.
7. Le touriste modifie ses choix ou sélectionne un autre hôtel.

A2 : Échec du paiement (étape 6)

7. Le système affiche un message d'erreur de paiement.
8. Le touriste peut réessayer ou annuler la réservation.

Postconditions :

- Une réservation est confirmée et enregistrée dans le système.

Titre : Transport

Résumé :

Le touriste connecté peut consulter des options de transport pour ses déplacements. Le système affiche des moyens de transport gratuits ou payants avec les détails du trajet. Si le service est payant, le paiement est effectué avant la confirmation de la réservation.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté.
- Le touriste possède un itinéraire ou une destination sélectionnée.

Scénario nominal :

1. Le touriste consulte un itinéraire ou choisit une destination.
2. Le système affiche les options de transport disponibles.
3. Le touriste sélectionne un moyen de transport.
4. Le système affiche les détails du trajet et le coût éventuel.
5. Si le transport est gratuit, le touriste accède directement à la navigation.
6. Si le transport est payant, le système lance le processus de paiement.
7. Après confirmation du paiement, le système affiche le résumé de réservation.
8. Le système affiche un message de confirmation du transport.

Enchaînements alternatifs :

- A1 : Aucun transport disponible (étape 2)
3. Le système informe qu'aucun moyen de transport n'est disponible.
 4. Le touriste peut modifier sa destination.

Postconditions :

- Une réservation est confirmée et enregistrée dans le système.

Titre : Visiter une Place Touristique**Résumé :**

Le touriste connecté peut consulter des informations culturelles, historiques et pratiques sur un lieu touristique. Il peut explorer la fiche du lieu, l'ajouter à son itinéraire, partager son expérience ou réserver un guide touristique.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'utilisateur est connecté.
- Le touriste a sélectionné un lieu touristique.

Scénario nominal :

1. Le touriste sélectionne un lieu touristique.
2. Le système affiche la fiche culturelle enrichie du lieu.
3. Le touriste consulte les informations et médias disponibles.
4. Le touriste peut ajouter le lieu à son itinéraire, commenter ou réserver un guide.
5. Le système enregistre l'interaction du touriste.
6. Le touriste quitte la fiche ou continue son exploration.

Enchaînements alternatifs :

A1 : Informations indisponibles (étape 2)

3. Le système affiche un message indiquant que les informations sont temporairement indisponibles.
4. Le touriste retourne à la liste ou à la carte.

A2 : Erreur de chargement des médias (étape 2)

3. Le système affiche un message d'erreur de chargement.
4. Le touriste peut actualiser la page ou continuer sans les médias.

A2 : Lieu non référencé (étape 1)

2. Le système informe que le lieu n'est pas encore intégré.
3. Le touriste peut proposer ce lieu à l'équipe.

Extension : Guide touristique

- Le touriste peut réserver un guide depuis la fiche du lieu.
- Le système affiche les guides disponibles avec horaires et tarifs.
- Après paiement, la réservation du guide est confirmée et ajoutée à l'itinéraire.

Postconditions :

- Le touriste a consulté les informations détaillées du lieu.
- Le lieu peut être ajouté à un itinéraire ou associé à une réservation.

Titre : Paiement**Résumé :**

Le système permet au touriste de payer un service réservé (restaurant, hôtel, activité ou guide) via différents modes de paiement sécurisés.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- Le touriste a sélectionné un service nécessitant un paiement.

Scénario nominal :

1. Le système lance le processus de paiement.
2. Le système affiche les modes de paiement disponibles.
3. Le touriste choisit un mode de paiement.
4. Le système traite la transaction.
5. La banque valide le paiement.
6. Le système confirme la réservation.
7. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Paiement refusé (étape 6)

7. Le système affiche un message indiquant l'échec du paiement.
8. Le touriste peut réessayer ou annuler.

A2 : Problème de connexion bancaire (étape 5)

6. Le système affiche un message d'erreur de connexion.
7. Le touriste peut réessayer plus tard.

Postconditions :

- Le paiement est validé et le service réservé est confirmé.

Titre : Gérer les Réservations

Résumé :

L'admin peut consulter et gérer les réservations des touristes en les confirmant, modifiant ou annulant.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'admin est connecté à son espace professionnel.

Scénario nominal :

1. L'admin accède au tableau de bord.
2. Le système affiche les réservations récentes.
3. L'admin sélectionne une réservation.
4. Le système affiche les détails de la réservation.
5. L'admin confirme, modifie ou annule la réservation.
6. Le système enregistre les modifications et notifie le touriste.
7. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Réservation déjà traitée (étape 5)

6. Le système indique que la réservation ne peut plus être modifiée.
7. L'admin consulte l'historique.

A2 : Échec de notification (étape 6)

7. Le système informe que la notification n'a pas été envoyée.
8. L'admin contacte le client autrement.

Postconditions :

- Les réservations sont mises à jour dans le système.

Titre : Analyser les Statistiques

Résumé :

L'administrateur peut consulter des rapports statistiques sur l'utilisation de l'application afin d'analyser les réservations et les performances des services.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'administrateur est connecté à l'interface d'administration.

Scénario nominal :

1. L'administrateur ouvre la section « Statistiques ».
2. Le système affiche les types de rapports disponibles.
3. L'administrateur sélectionne un rapport.
4. Le système génère les statistiques.
5. Le système affiche les résultats sous forme de tableaux et graphiques.
6. L'administrateur peut exporter les rapports.

Enchaînements alternatifs :

A1 : Données insuffisantes (étape 4)

5. Le système affiche un message indiquant qu'il manque des données.
6. L'administrateur ajuste les paramètres.

A2 : Erreur de génération (étape 4)

5. Le système affiche un message d'erreur.
6. L'administrateur peut réessayer plus tard.

Postconditions :

- Les rapports statistiques sont générés et affichés.

Titre : Gérer les Contenus

Résumé :

L'administrateur peut ajouter, modifier ou supprimer les contenus culturels et touristiques de l'application.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'administrateur est connecté à son espace d'administration.

Scénario nominal :

1. L'administrateur accède à la gestion des contenus.
2. Le système affiche les éléments disponibles.
3. L'administrateur sélectionne un contenu à gérer.
4. Le système affiche le formulaire d'édition.
5. L'administrateur modifie les informations nécessaires.
6. L'administrateur enregistre les changements.
7. Le système valide et met à jour les données.
8. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Données incomplètes (étape 6)

7. Le système demande de compléter les champs obligatoires.
8. L'administrateur corrige les informations.

A2 : Échec de sauvegarde (étape 6)

7. Le système affiche un message d'erreur.
8. L'administrateur peut réessayer ou annuler.

Postconditions :

- Les contenus de l'application sont mis à jour.

Titre : Mettre à Jour les Disponibilités

Résumé :

L'admin peut gérer les disponibilités des chambres, tables ou activités afin de contrôler les réservations.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'admin est connecté à son espace professionnel.

Scénario nominal :

1. L'admin accède à la gestion des disponibilités.
2. Le système affiche les créneaux disponibles.
3. L'admin sélectionne un créneau.
4. L'admin modifie son statut.
5. L'admin enregistre les changements.
6. Le système met à jour les disponibilités.
7. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Conflit avec une réservation existante (étape 4)

5. Le système indique qu'une réservation active existe déjà.
6. L'admin choisit un autre créneau ou annule la réservation concernée.

Postconditions :

- Les disponibilités sont mises à jour dans le système.

Titre : Mettre à Jour le Catalogue**Résumé :**

L'admin peut modifier les informations des services proposés comme les descriptions, prix ou photos.

Acteurs :

- Touriste

Date de création : 10/12/2025

Date de mise à jour : —

Auteur : Hamza Chdaoui.

Préconditions :

- L'admin est connecté.

Scénario nominal :

1. L'admin accède à la section « Mon Catalogue ».
2. Le système affiche la liste des services.
3. L'admin sélectionne un service à modifier.
4. Le système affiche un formulaire pré-rempli.
5. L'admin modifie les informations nécessaires.
6. L'admin enregistre les modifications.
7. Le système valide et met à jour le catalogue.
8. Le système affiche un message de confirmation.

Enchaînements alternatifs :

A1 : Données invalides (étape 6)

7. Le système affiche un message d'erreur précis.
8. L'admin corrige les données puis réessaie.

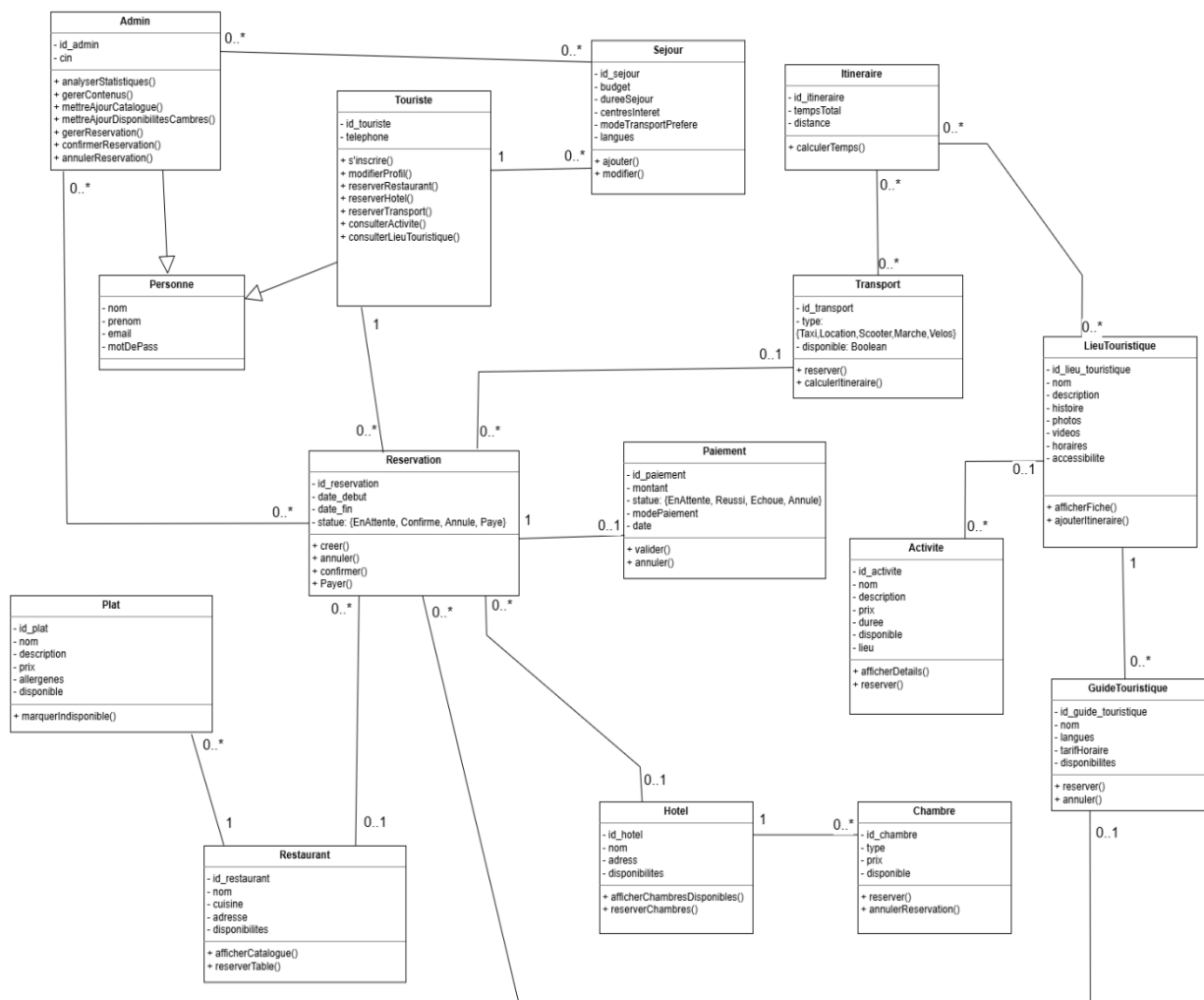
Postconditions :

- Les informations du catalogue sont mises à jour et visibles par les touristes.

2. MODÉLISATION TECHNIQUE

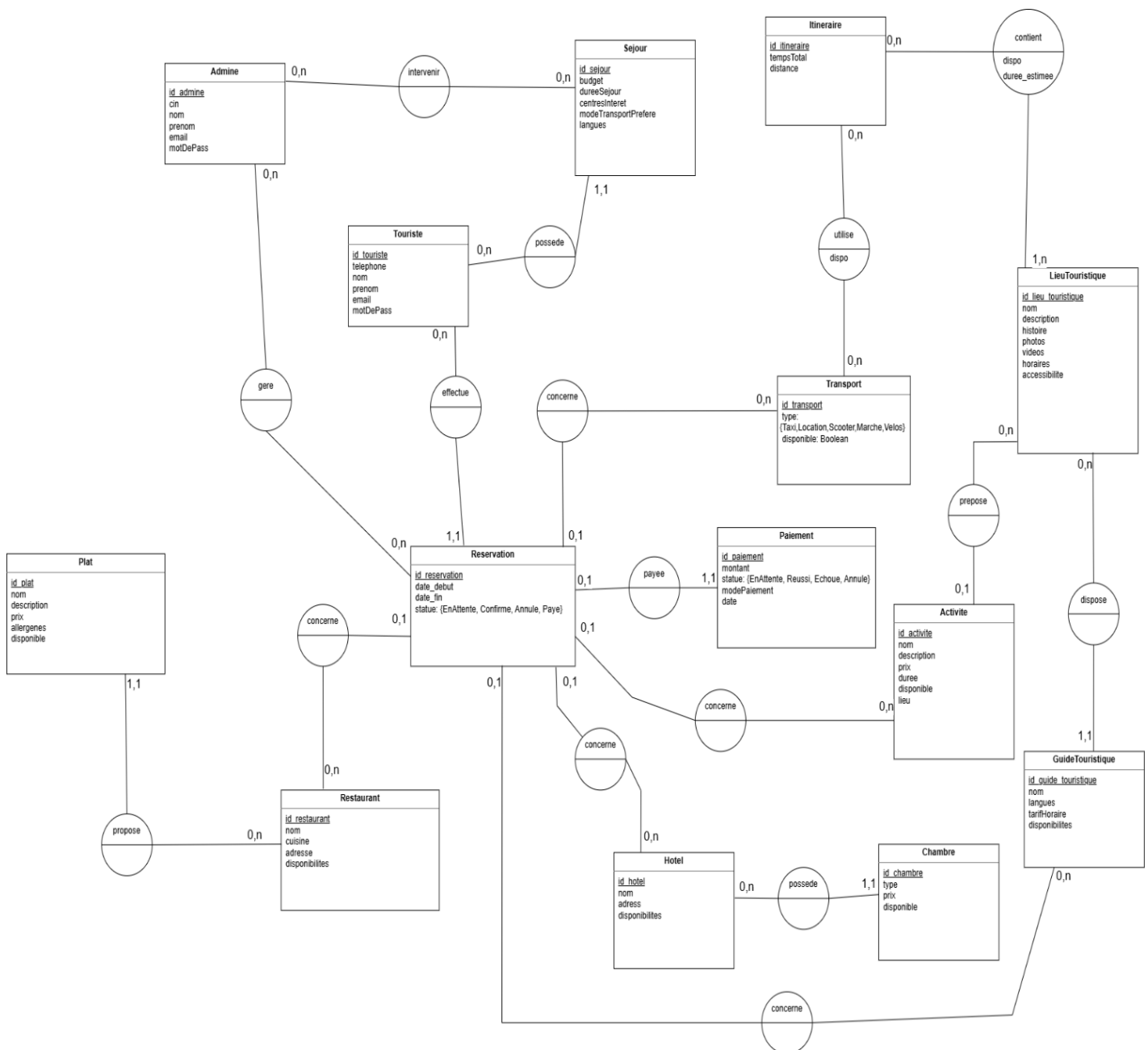
2.1 Diagramme de Classes UML

Le diagramme de classes constitue la représentation structurale centrale du système d'information Elyora. Il modélise l'ensemble des entités du domaine, leurs attributs, leurs méthodes ainsi que les relations qui les unissent (associations, agrégations, compositions et héritages). L'architecture orientée objet adoptée garantit une forte cohésion interne des modules et un faible couplage entre eux, favorisant ainsi la maintenabilité et l'évolutivité du système.



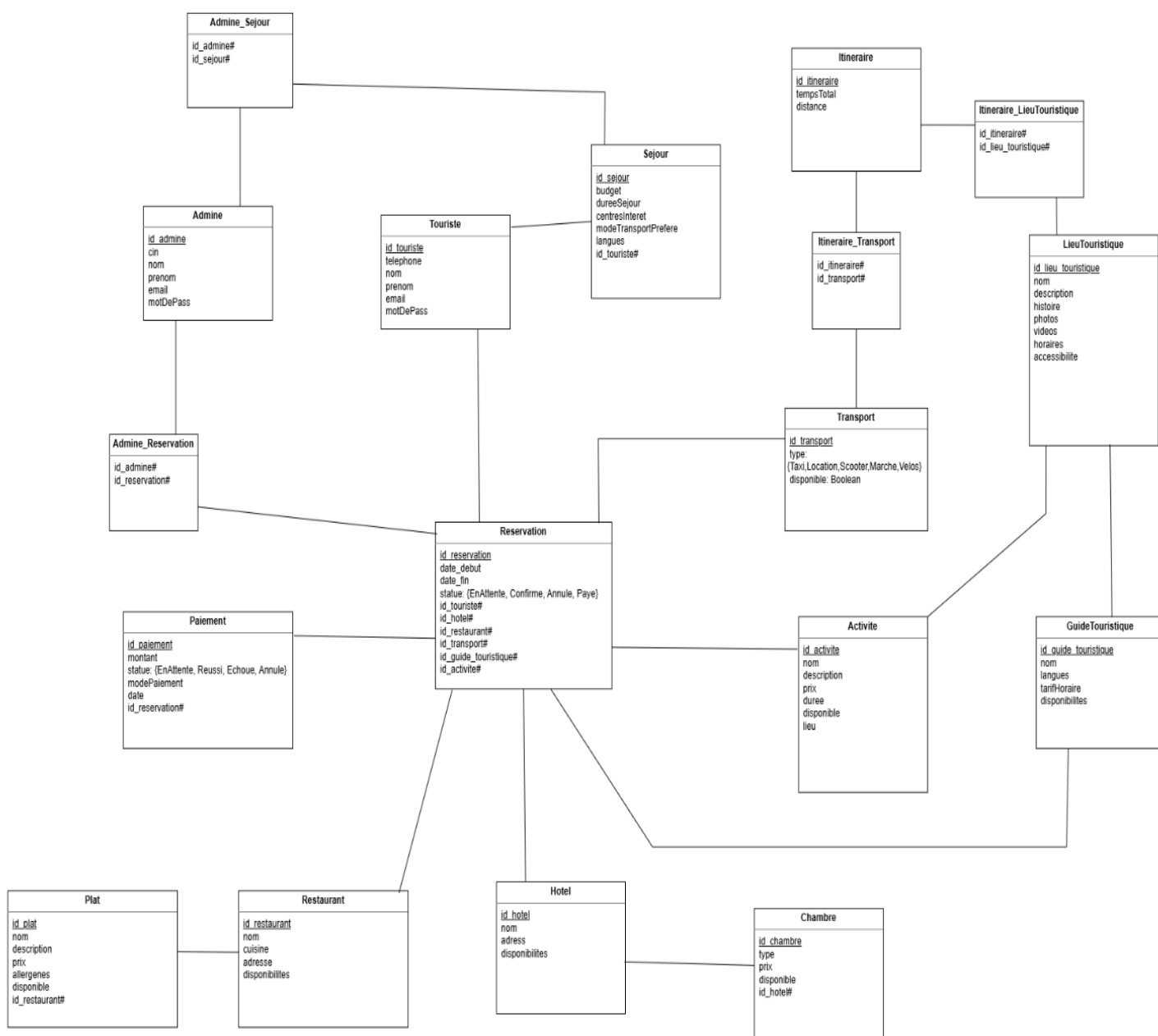
2.2 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données (MCD) offre une représentation abstraite et indépendante de toute implémentation technique des informations gérées par le système. Il identifie les entités principales (Touriste, Réservation, Paiement, LieuTouristique, etc.) ainsi que leurs associations et cardinalités, constituant ainsi le fondement logique sur lequel repose l'architecture de la base de données.



2.3 Modèle Logique de Données (MLD)

Le Modèle Logique de Données (MLD) constitue la traduction du MCD en un schéma adapté au Système de Gestion de Base de Données Relationnelle (SGBDR) retenu. Il précise les tables, les clés primaires et étrangères, ainsi que les contraintes d'intégrité référentielle qui assurent la cohérence des données au niveau physique.



2.4 Script de Création de la Base de Données

Le script SQL ci-après présente la définition des principales tables de la base de données du système Elyora, conformément au MLD validé. Il a été conçu pour un SGBDR MySQL et intègre l'ensemble des contraintes d'intégrité (clés primaires, clés étrangères, contraintes de vérification) nécessaires à la fiabilité des données.

```
CREATE DATABASE IF NOT EXISTS agence_voyage;
USE agence_voyage;

CREATE TABLE Admine (
    id_admine INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    prenom VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    motDePasse VARCHAR(255) NOT NULL,
    cin VARCHAR(255) NOT NULL
);

CREATE TABLE Touriste (
    id_touriste INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    prenom VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    motDePasse VARCHAR(255) NOT NULL,
    telephone VARCHAR(20) NOT NULL
);

CREATE TABLE Sejour (
    id_sejour INT AUTO_INCREMENT PRIMARY KEY,
    id_touriste INT UNIQUE NOT NULL,
    budget DECIMAL(10,2),
    dureeSejour INT,
    centresInteret TEXT,
    modeTransportPrefere VARCHAR(100),
    langues VARCHAR(255),
    FOREIGN KEY (id_touriste) REFERENCES Touriste(id_touriste) ON DELETE SET
    NULL
);

CREATE TABLE Admine_Reservation (
    id_admine INT NOT NULL,
    id_reservation INT NOT NULL,
    PRIMARY KEY (id_admine, id_reservation),
    FOREIGN KEY (id_admine) REFERENCES Admine(id_admine) ON DELETE SET NULL,
    FOREIGN KEY (id_reservation) REFERENCES Reservation(id_reservation) ON
    DELETE SET NULL
);

CREATE TABLE Admine_Sejour (
    id_admine INT NOT NULL,
    id_sejour INT NOT NULL,
    PRIMARY KEY (id_admine, id_sejour),
    FOREIGN KEY (id_admine) REFERENCES Admine(id_admine) ON DELETE SET NULL,
    FOREIGN KEY (id_sejour) REFERENCES Sejour(id_sejour) ON DELETE SET NULL
);

CREATE TABLE Itineraire (
    id_itineraire INT AUTO_INCREMENT PRIMARY KEY,
    tempsTotal DECIMAL(10,2),
    distance DECIMAL(10,2)
```

```
);

CREATE TABLE Itineraire_LieuTouristique (
    id_itineraire INT NOT NULL,
    id_lieu_touristique INT NOT NULL,
    type VARCHAR(50),
    dispo BOOLEAN DEFAULT TRUE,
    duree_estimee DECIMAL(10,2),
    PRIMARY KEY (id_itineraire, id_lieu_touristique),
    FOREIGN KEY (id_itineraire) REFERENCES Itineraire(id_itineraire) ON DELETE
SET NULL,
    FOREIGN KEY (id_lieu_touristique) REFERENCES
LieuTouristique(id_lieu_touristique) ON DELETE SET NULL
);

CREATE TABLE LieuTouristique (
    id_lieu_touristique INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    description TEXT,
    histoire TEXT,
    photos TEXT,
    videos TEXT,
    horaires VARCHAR(255),
    accessibilite VARCHAR(255)
);

CREATE TABLE Itineraire_Transport (
    id_itineraire INT NOT NULL,
    id_transport INT NOT NULL,
    type VARCHAR(50),
    dispo BOOLEAN DEFAULT TRUE,
    PRIMARY KEY (id_itineraire, id_transport),
    FOREIGN KEY (id_itineraire) REFERENCES Itineraire(id_itineraire) ON DELETE
SET NULL,
    FOREIGN KEY (id_transport) REFERENCES Transport(id_transport) ON DELETE SET
NULL
);

CREATE TABLE Transport (
    id_transport INT AUTO_INCREMENT PRIMARY KEY,
    type ENUM('Taxi', 'Location', 'Scooter', 'Marche', 'Velos') NOT NULL,
    disponibilite BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE GuideTouristique (
    id_guide_touristique INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    langues VARCHAR(255),
    tarifHoraire DECIMAL(10,2) NOT NULL,
    disponibilites BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE Hotel (
    id_hotel INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    adresse VARCHAR(255) NOT NULL,
    disponibilites BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE Chambre (
    id_chambre INT AUTO_INCREMENT PRIMARY KEY,
    id_hotel INT NOT NULL,
    type VARCHAR(100) NOT NULL,
    prix DECIMAL(10,2) NOT NULL,
    disponible BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (id_hotel) REFERENCES Hotel(id_hotel) ON DELETE CASCADE
);

CREATE TABLE Restaurant (
```

```

    id_restaurant INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    cuisine VARCHAR(100),
    adresse VARCHAR(255) NOT NULL,
    disponibles BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE Plat (
    id_plat INT AUTO_INCREMENT PRIMARY KEY,
    id_restaurant INT NOT NULL,
    description TEXT,
    prix DECIMAL(10,2) NOT NULL,
    allergenes TEXT,
    disponibles BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (id_restaurant) REFERENCES Restaurant(id_restaurant) ON DELETE
SET NULL
);

CREATE TABLE Activite (
    id_activite INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    description TEXT,
    prix DECIMAL(10,2) NOT NULL,
    duree VARCHAR(50),
    disponible BOOLEAN NOT NULL DEFAULT TRUE,
    lieu VARCHAR(255)
);

CREATE TABLE Reservation (
    id_reservation INT AUTO_INCREMENT PRIMARY KEY,
    id_touriste INT NOT NULL,
    date_debut DATETIME NOT NULL,
    date_fin DATETIME NOT NULL,
    statut ENUM('EnAttente', 'Confirme', 'Annule', 'Paye') NOT NULL DEFAULT
'EnAttente',
    id_hotel int(11) DEFAULT NULL,
    id_restaurant int(11) DEFAULT NULL,
    id_transport int(11) DEFAULT NULL,
    id_guide_touristique int(11) DEFAULT NULL,
    id_activite int(11) DEFAULT NULL,
    FOREIGN KEY (id_touriste) REFERENCES Touriste(id_touriste) ON DELETE SET
NULL,
    FOREIGN KEY (id_hotel) REFERENCES Hotel(id_hotel) ON DELETE SET NULL,
    FOREIGN KEY (id_restaurant) REFERENCES Restaurant(id_restaurant) ON DELETE
SET NULL,
    FOREIGN KEY (id_transport) REFERENCES Transport(id_transport) ON DELETE SET
NULL,
    FOREIGN KEY (id_guide_touristique) REFERENCES
GuideTouristique(id_guide_touristique) ON DELETE SET NULL,
    FOREIGN KEY (id_activite) REFERENCES Activite(id_activite) ON DELETE SET
NULL,
);

CREATE TABLE Paiement (
    id_paiement INT AUTO_INCREMENT PRIMARY KEY,
    id_reservation INT NOT NULL,
    montant DECIMAL(10,2) NOT NULL,
    statut ENUM('EnAttente', 'Reussi', 'Echoue', 'Annule') NOT NULL DEFAULT
'EnAttente',
    modePaiement VARCHAR(50) NOT NULL,
    date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_reservation) REFERENCES Reservation(id_reservation) ON
DELETE CASCADE
);

```


2.5 Concepts de la Programmation Orientée Objet

La programmation orientée objet (POO) constitue le paradigme fondateur sur lequel repose l'architecture logicielle d'Elyora. Elle permet de structurer les programmes en organisant le code autour d'objets représentant des entités du monde réel, favorisant ainsi la réutilisabilité, la modularité et la maintenabilité du code.

Les quatre principes fondamentaux de la POO, tels qu'ils ont été appliqués dans le cadre de ce projet, sont les suivants :

- Encapsulation : regrouper les données (attributs) et les comportements (méthodes) au sein d'une même classe, tout en contrôlant précisément les droits d'accès depuis l'extérieur.
- Héritage : créer de nouvelles classes à partir de classes existantes afin de réutiliser leurs propriétés et leurs comportements, tout en les spécialisant. Exemple : les sous-classes PaiementCarte, PaiementCash et PaiementEnLigne héritent de la classe abstraite Paiement.
- Polymorphisme : permettre à une même interface ou méthode de se comporter différemment selon le type d'objet sur lequel elle est invoquée, offrant ainsi une grande flexibilité architecturale.
- Abstraction : simplifier la complexité en ne proposant aux utilisateurs d'une classe que les éléments essentiels à leur interaction, dissimulant les détails d'implémentation.

Ces principes ont été mis en œuvre de manière cohérente dans l'ensemble de la conception UML d'Elyora, garantissant un code plus lisible, plus évolutif et plus facile à maintenir sur le long terme.

2.6 Architecture Technique Globale

Le système Elyora repose sur une architecture tri-technologique composée de trois interfaces complémentaires, chacune dédiée à un rôle spécifique et implémentée dans le langage le plus adapté à ses exigences fonctionnelles. Ces trois composants partagent une base de données MySQL commune (agence_voyage), garantissant la cohérence des données à travers l'ensemble du système. Le patron de conception MVC (Modèle-Vue-Contrôleur) est appliqué de manière homogène dans les trois technologies, démontrant la maîtrise de ce patron indépendamment du langage d'implémentation.

2.6.1 Vue d'ensemble des Trois Composants

Site Web — Elyora	Console C++	Console Java
Interface Touriste	Admin — Operations	Admin — Rapports
<i>PHP (OOP) · PDO · Sessions</i> <i>HTML + CSS + JavaScript</i> <i>13 fichiers · MySQL</i> <i>agence_voyage</i>	<i>Structs.h · Database.h/.cpp</i> <i>Admin.h/.cpp · main.cpp</i> <i>Gestion Rooms, Réservations,</i> <i>Orders</i>	<i>RapportReservation ·</i> <i>StatistiquesRapport</i> <i>StatistiquesService · Database</i> <i>MenuAdmin · ConsoleHelper</i>
Public · CRUD complet	En mémoire · CRUD local	Fichiers · Statistiques

Base de Donnée partagée : MySQL — agence_voyage · Les 3 interfaces lisent/écrivent dans la même base de données. Cohérence garantie.

2.6.2 Patron de Conception MVC — Appliqué dans les 3 Technologies

Le patron MVC structure chaque composant de manière identique, quelle que soit la technologie utilisée. La Vue prend en charge l'interface utilisateur, le Contrôleur gère la logique métier, et le Modèle assure l'accès aux données. Cette homogénéité architecturale garantit la maintenabilité et la lisibilité du code à travers les trois implémentations.

Couche	PHP (Site Web)	C++ (Console)	Java (Console)
Vue (View)	HTML + CSS + JS (13 pages)	main.cpp (menus cin/cout)	MenuAdmin + ConsoleHelper
Contrôleur	config.php + pages PHP	Admin.cpp (login, CRUD)	StatistiquesService
Modèle (Data)	PDO → MySQL agence_voyage	Database.cpp + Structs.h	RapportReservation · StatistiquesRapport

2.6.3 Justification du Choix Technologique

PHP + HTML/CSS/JS — Le touriste accède à l'application depuis un navigateur web : PHP constitue la technologie native du web. PDO sécurise chaque requête SQL (protection anti-injection). Les sessions PHP maintiennent l'état de connexion entre les 13 pages, et JavaScript enrichit l'expérience utilisateur côté client.

C++ — C++ est utilisé pour démontrer la maîtrise des structs (Chambre, Réservation, MenuItem, Order) et des classes (Admin, Database). La gestion directe en mémoire par vecteurs STL illustre les bases de la POO sans dépendance externe.

Java — Java satisfait l'exigence de gestion de fichiers : `RapportReservation` et `StatistiquesRapport` sont sérialisés en CSV. `StatistiquesService` calcule les agrégats et génère les rapports. Les quatre packages (`model` / `service` / `view` / `database`) forment une architecture MVC stricte et propre.

3. GESTION DE PROJET

3.1 Méthodologie de Développement

La conduite du projet Elyora a été organisée selon une approche structurée combinant la méthode Agile Scrum pour la gestion itérative du développement et le Cycle en Y pour la structuration de la conception technique et fonctionnelle.

3.1.1 Le Manifeste Agile et ses Valeurs Fondatrices

L'approche Agile, formalisée par le Manifeste Agile de 2001, repose sur quatre valeurs fondamentales qui ont guidé l'organisation du travail au sein de l'équipe :

- Les individus et leurs interactions priment sur les processus et les outils.
- Un logiciel fonctionnel est prioritaire sur une documentation exhaustive.
- La collaboration avec le client et les parties prenantes est privilégiée sur la négociation contractuelle.
- La réactivité face au changement est valorisée sur le suivi d'un plan figé.

3.1.2 Le Cycle en Y — Double Branche de Conception

Le Cycle en Y structure le développement du projet Elyora selon deux branches parallèles et convergentes :

- Branche fonctionnelle (Quoi ?) : capture des besoins fonctionnels des touristes, prestataires et administrateurs ; analyse de quinze cas d'utilisation avec scénarios nominaux et alternatifs ; spécification via les diagrammes de contexte et de cas d'utilisation.
- Branche technique (Comment ?) : architecture technique via le diagramme de classes UML, le MCD et le MLD ; conception détaillée des sous-classes de paiement et de la gestion des statuts de réservation ; réalisation et tests d'intégration.

Les deux branches progressent en parallèle et convergent vers un système validé et déployé — l'application Elyora livrée, testée et mise en production.

3.2 Organisation Scrum de l'Équipe

L'organisation de l'équipe Elyora s'est structurée selon les rôles Scrum standard :

- ▶ Product Owners : M. Mani, M. Echchadli, M. Serghini et M. Mouhib définissent les priorités du backlog produit, valident les livrables et orientent les choix fonctionnels.
- ▶ Scrum Master : Hamza Chdaoui facilite les cérémonies Scrum (Daily Stand-up, Sprint Review, Rétrospective), lève les obstacles organisationnels et garantit l'application des bonnes pratiques agiles.
- ▶ Équipe de développement : Salah-Eddine El-Ghazi, Souad Harchaoui, Ayoub Arabat et Hamza Chdaoui assurent le développement des fonctionnalités sprint par sprint (backend, frontend, base de données, tests).

3.3 Product Backlog et Sprints de Développement

Le développement d'Elyora a été découpé en cinq sprints itératifs, chacun livrant un module fonctionnel validé :

- ▶ Sprint 1 : Inscription et gestion du profil touriste.
- ▶ Sprint 2 : Génération d'itinéraire et consultation des sites touristiques.
- ▶ Sprint 3 : Réservation de restaurant et d'hôtel.
- ▶ Sprint 4 : Module de paiement (carte bancaire, espèces, paiement en ligne).
- ▶ Sprint 5 : Module d'administration.

3.4 Cycle de Vie du Projet — Six Phases Clés

Le cycle de vie du projet Elyora s'articule autour de six phases successives, de l'expression du besoin initial jusqu'à la maintenance évolutive :

Phase 01	Expression des Besoins	Identification des besoins des touristes et prestataires locaux, définition des objectifs du système Elyora et identification des acteurs.
Phase 02	Analyse	Étude du problème, élaboration du diagramme de contexte, identification des cas d'utilisation et rédaction des scénarios nominaux.
Phase 03	Conception	Propositions de solutions architecturales, élaboration du diagramme de classes, du MCD/MLD et de l'architecture générale du SI.
Phase 04	Réalisation	Développement par sprints Scrum : backend API REST, frontend mobile (UI/UX), intégration des modules de paiement.
Phase 05	Mise en Œuvre	Livraison de l'application, formation des prestataires, déploiement cloud et tests utilisateurs en conditions réelles.
Phase 06	Maintenance	Maintenance corrective (correction de bogues) et évolutive (intégration de nouvelles fonctionnalités), suivi des indicateurs de performance.

3.5 Planification — Diagramme de Gantt

La planification du projet Elyora s'est appuyée sur un diagramme de Gantt couvrant la période de novembre 2025 à juin 2026, permettant une gestion rigoureuse des délais et des dépendances entre tâches.

Tâche	Membres	NOV	DÉC	JAN	FÉV	MAR	AVR	MAI	JUN
Expression des besoins	Équipe	•							
Analyse & Diagramme de contexte	Hamza, Souad	•	•						
Conception (classes, MCD/MLD)	Salah, Ayoub		•	•					
Réalisation — Module Touriste	Souad, Ayoub			•	•	•			
Réalisation — Réservation/Paiement	Salah, Hamza				•	•	•		
Réalisation — Module Admin	Équipe					•	•		
Tests & Intégration	Hamza, Ayoub						•	•	
Mise en œuvre & Documentation	Équipe							•	•

3.6 Importance du Travail en Équipe

La réussite du projet Elyora s'est appuyée sur une dynamique collaborative structurée et efficace. Le travail en équipe a constitué un levier essentiel, tant sur le plan technique que humain, selon les dimensions suivantes :

- Complémentarité des compétences : chaque membre a apporté ses points forts spécifiques (modélisation UML, développement backend, conception frontend, administration de bases de données), garantissant ainsi une qualité globale supérieure à ce qu'un travail individuel aurait pu produire.
- Apprentissage collaboratif : la confrontation des idées et la justification des choix de conception (structure des classes, relations du modèle de données, scénarios alternatifs) ont consolidé la compréhension collective des concepts.
- Responsabilisation et gestion de projet : la planification rigoureuse, l'organisation de réunions régulières et la fixation de jalons intermédiaires ont préparé l'équipe aux exigences du monde professionnel.
- Amélioration de la communication technique : la rédaction collective de documents, la présentation structurée des idées et l'écoute active ont renforcé les compétences interpersonnelles essentielles à l'ingénieur.
- Réduction des biais individuels : le regard critique des pairs a permis de détecter les incohérences de conception (acteurs oubliés, associations manquantes, scénarios non traités) et de les corriger avant les phases de développement.

Une planification rigoureuse et une collaboration structurée constituent les fondements de la réussite du projet Elyora — transformant la vision en un livrable concret, maîtrisé et de qualité.

CHAPITRE III — Réalisation

3.1 Site Web – Elyora — Interface - Touriste

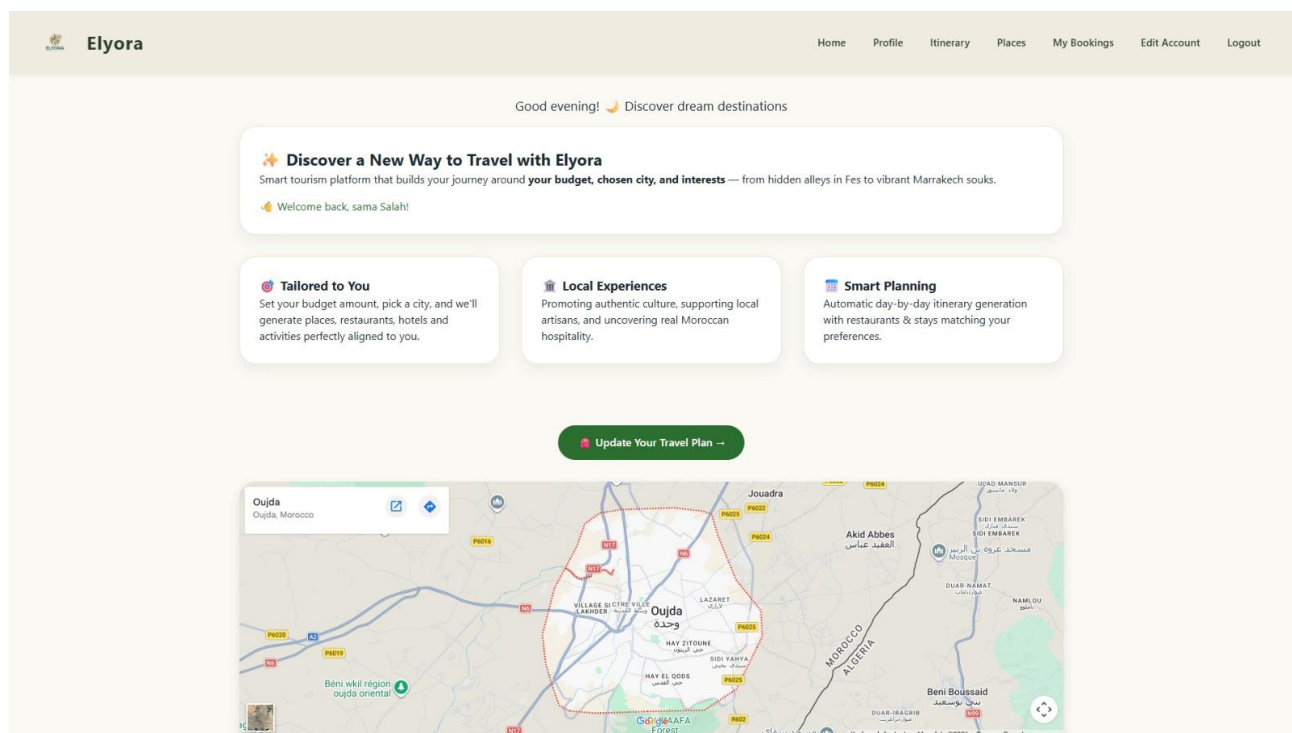
Le site web « Elyora » utilise PHP, HTML, CSS et JavaScript pour créer une interface touristique interactive et accessible depuis un navigateur.

PHP gère la logique de l'application et la communication avec la base de données, tandis que PDO sécurise les requêtes SQL contre les injections.

Les sessions PHP permettent de maintenir la connexion du touriste entre les différentes pages du site.

HTML et CSS assurent la structure et le design des pages, alors que JavaScript améliore l'expérience utilisateur avec des interactions dynamiques et une navigation plus fluide.

3.1.1 Page d'accueil du site Elyora



Cette interface représente la page d'accueil principale du site touristique **Elyora** après la connexion de l'utilisateur.

Elle accueille le touriste avec un message personnalisé et présente les principales fonctionnalités de la plateforme.

Le système permet à l'utilisateur de découvrir des destinations, de planifier son voyage et d'obtenir des recommandations adaptées à son budget et à ses centres d'intérêt.

La page contient plusieurs sections interactives :

- une présentation générale du service ;
- des cartes informatives sur les expériences locales et la planification intelligente ;
- un bouton permettant de mettre à jour le plan de voyage ;
- une carte géographique interactive affichant la destination choisie.

Cette interface a été développée avec **HTML** et **CSS** pour la structure et le design, tandis que **JavaScript** assure l'interactivité de la carte et la fluidité de navigation. Les données affichées sont récupérées grâce à **PHP** et à une base de données sécurisée via **PDO**.

3.1.2 Page d'inscription (Register)

The screenshot shows the 'Create Your Account' registration form on the Elyora website. The form is centered on a light beige background. At the top left of the page is the 'EL Elyora' logo, and at the top right are links for 'Home', 'Login', and 'Register'. The form itself has a title 'Create Your Account' with a small icon. It contains several input fields: 'Last Name *', 'First Name' (with a placeholder 'Please fill out this field.'), 'Email Address *', 'Telephone *', 'Password * (min 8 characters)', and 'Confirm Password *'. Each field is represented by a rounded rectangular input box. At the bottom of the form is a green button labeled 'Register →'.

Cette page permet aux nouveaux utilisateurs de créer un compte sur la plateforme Elyora.

Le formulaire demande plusieurs informations personnelles comme :

- le nom ;
- le prénom ;
- l'adresse e-mail ;
- le numéro de téléphone ;
- le mot de passe.

Le système vérifie automatiquement les champs obligatoires afin d'éviter les erreurs de saisie et d'assurer la validité des données.

Les informations sont ensuite envoyées au serveur à l'aide de **PHP**, puis enregistrées dans la base de données via **PDO**, ce qui garantit la sécurité contre les injections SQL.

L'interface a été conçue avec **HTML** et **CSS** pour offrir une apparence moderne et simple d'utilisation, tandis que **JavaScript** améliore la validation des formulaires et l'expérience utilisateur.

3.1.3 Page de connexion (Login)

The screenshot shows the Elyora login interface. At the top, there is a header bar with the Elyora logo on the left and navigation links for 'Home', 'Login', and 'Register' on the right. The main content area is a light beige color. In the center, there is a white rounded rectangle representing the login form. The form has a title 'Login to Your Account' with a small lock icon. Below the title, there are two input fields: 'Email Address' and 'Password'. Below these fields is a green button with the text 'Login →'. At the bottom of the form, there is a link that says 'Don't have an account? Register here'.

Cette interface correspond à la page de connexion des utilisateurs enregistrés.

Elle permet au touriste d'accéder à son espace personnel en saisissant son adresse e-mail et son mot de passe.

Après la vérification des informations par **PHP**, le système crée une **session PHP** afin de maintenir la connexion active pendant toute la navigation sur le site.

Cette fonctionnalité permet à l'utilisateur d'accéder à ses réservations, son itinéraire et ses préférences sans devoir se reconnecter à chaque page.

Le design de la page repose sur **HTML** et **CSS** pour une interface claire et ergonomique, tandis que **JavaScript** facilite les interactions et améliore la fluidité de l'expérience utilisateur.

3.1.4 Page des Préférences de Voyage

The screenshot displays the 'Your Travel Preferences' page within the Elyora application. The interface features a clean, modern design with a light beige background and a white central form. The form is titled 'Your Travel Preferences' and includes a welcome message: 'Welcome back, sama Salah!'. The form contains several input fields and a dropdown menu:

- Your Budget (USD) ***: A text input field with the value '200.00'.
- Budget in US Dollars (USD)**: A label for the budget field.
- Choose your destination city ***: A dropdown menu with the selected option 'Fes - Spiritual & artisanal heart'.
- Your Interests (select at least one)**: A section with four checkboxes: 'History', 'Food' (checked), 'Nature', and 'Art / Craft'.
- Preferred Transport ***: A dropdown menu with the selected option 'Walking'.
- Trip Duration (days) ***: A text input field with the value '3'.

At the bottom of the form is a green button labeled 'Save Travel Preferences'. The top navigation bar includes the Elyora logo and links for Home, Profile, Itinerary, Places, My Bookings, Edit Account, and Logout.

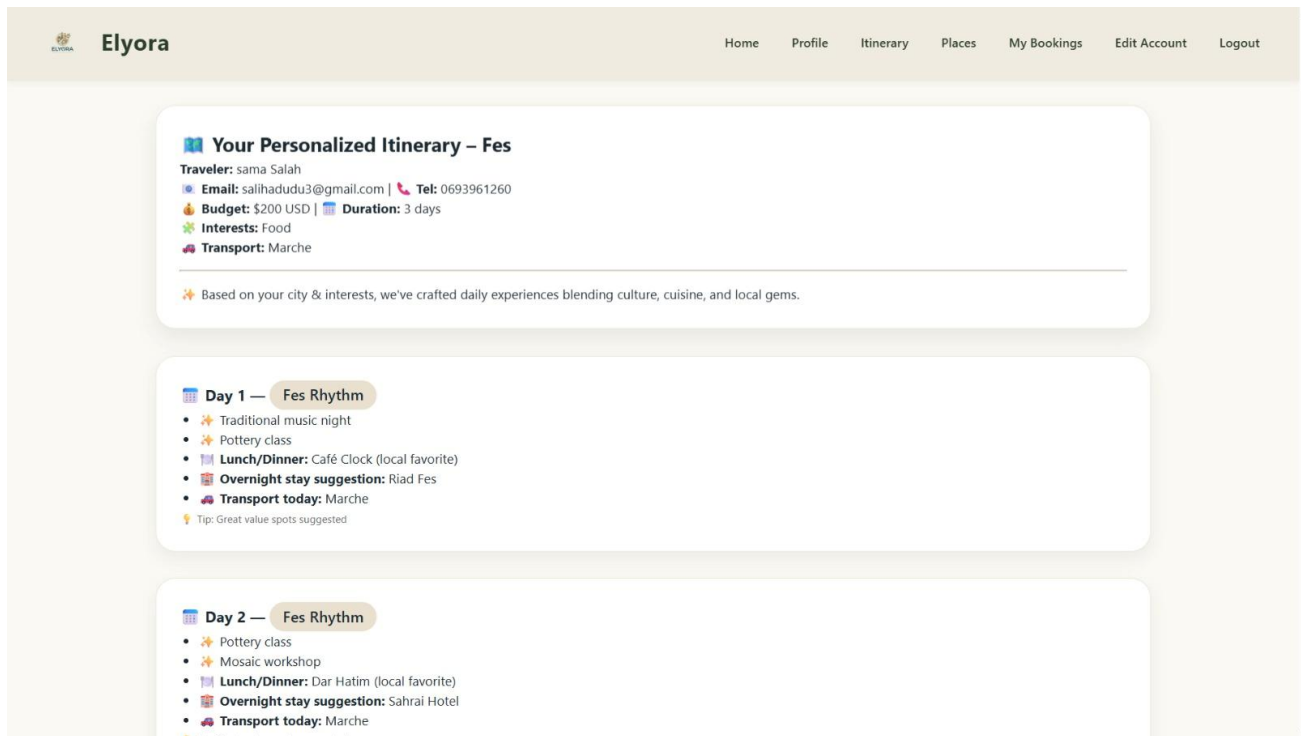
Cette interface permet à l'utilisateur de personnaliser son expérience en définissant ses critères de voyage. Le formulaire interactif recueille des informations essentielles telles que :

- **Le budget total** (en USD) ;
- **La ville de destination** (via un menu déroulant) ;
- **Les centres d'intérêt** (Histoire, Gastronomie, Nature, Art/Artisanat) via des cases à cocher ;

- **Le mode de transport préféré et la durée du séjour.**

Les données saisies sont traitées par **PHP** pour filtrer les activités correspondantes dans la base de données. L'utilisation de **JavaScript** permet une validation dynamique des champs, tandis que la structure en **HTML/CSS** assure une présentation claire et intuitive.

3.1.5 Page de l'itinéraire Personnalisé

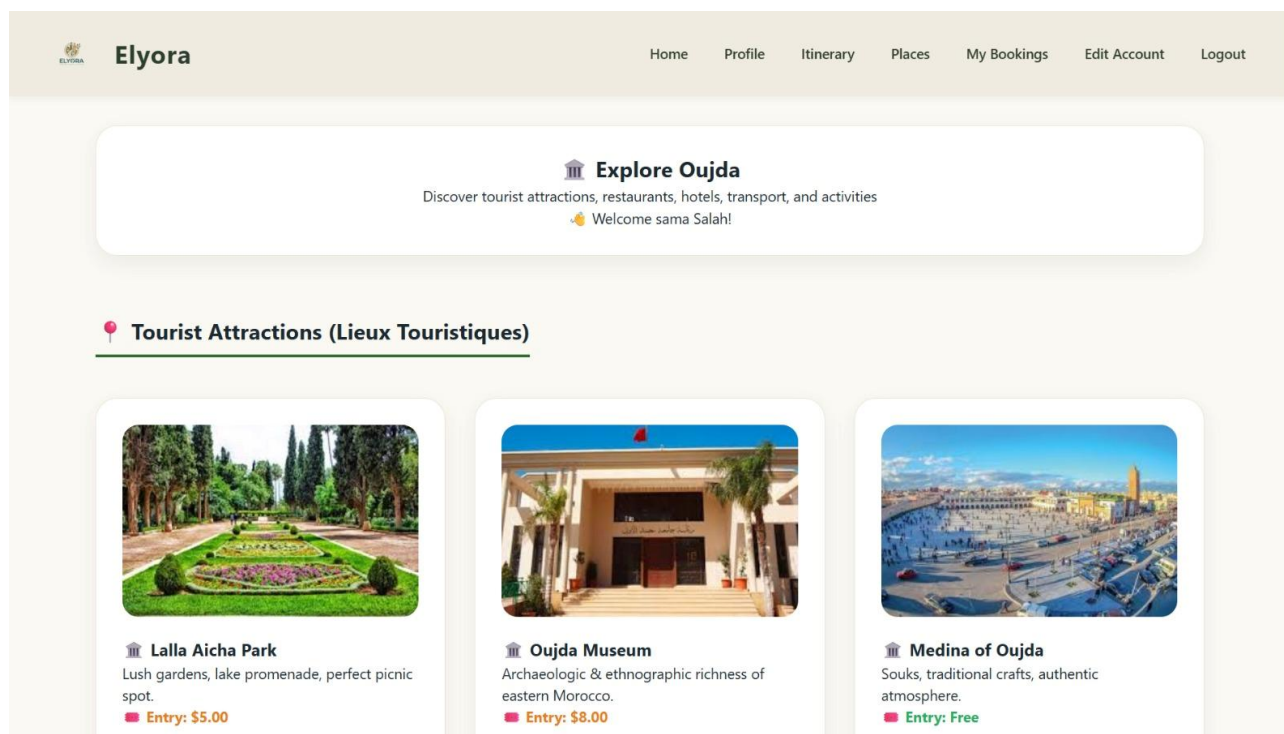


Cette page affiche le résultat de l'algorithme de planification basé sur les préférences de l'utilisateur. Elle présente un programme quotidien détaillé (Jour 1, Jour 2, etc.) comprenant :

- Des activités spécifiques (ateliers, cours de poterie, soirées musicales) ;
- Des suggestions de restaurants et d'hébergements (Riad, Hôtels) ;
- Le mode de transport choisi.

Le système utilise des requêtes **SQL via PDO** pour extraire les données pertinentes et les afficher dynamiquement. Le design épuré en **CSS** facilite la lecture du planning, offrant au touriste une vision globale de son séjour optimisé.

3.1.6 Page d'Exploration des Lieux (Destinations)



Cette interface sert de catalogue visuel pour les attractions touristiques des villes proposées (Fès, Oujda, Marrakech). Chaque lieu est présenté sous forme de **carte interactive (Card)** incluant :

- Une image représentative de haute qualité ;
- Une brève description historique ou culturelle ;
- Le prix d'entrée (ou la mention "Gratuit") ;
- Un bouton d'action pour réserver la visite.

Grâce à **PHP**, le contenu s'adapte dynamiquement selon la ville sélectionnée par l'utilisateur.

JavaScript gère les effets de survol et les transitions, tandis que les **sessions PHP** permettent de mémoriser les choix pour l'étape de réservation.

3.1.7 Page de Gestion des Réservations (My Bookings)

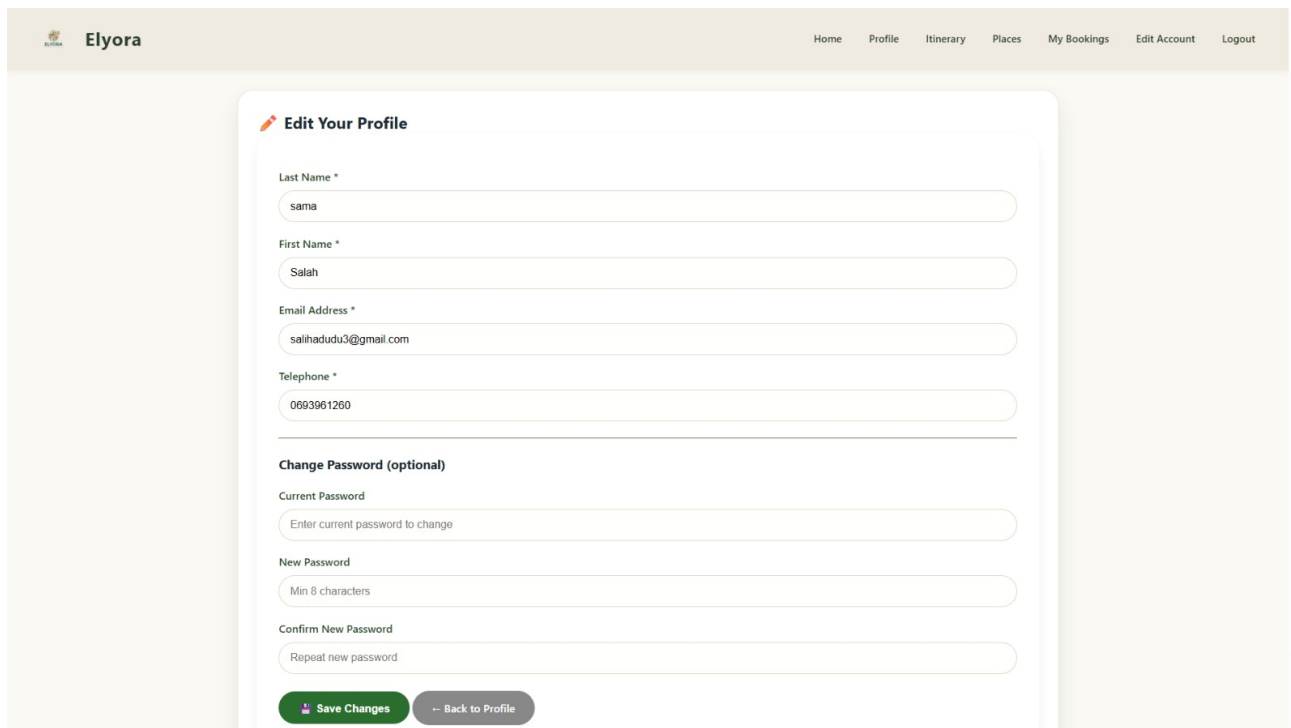
The screenshot displays the 'My Travel Bookings' page of the Elyora application. The page has a navigation bar at the top with links: Home, Profile, Itinerary, Places, My Bookings, Edit Account, and Logout. The main content area is titled 'My Travel Bookings' and includes a welcome message: 'Welcome back, sama Salah! Here are all your confirmed reservations.' Below this is a blue box with a 'School Project Demo Note' stating that PayPal payments work in demo mode and that all bookings are saved. The main booking details are shown in a white box with a red location pin icon, indicating a booking for Morocco. The booking was made on May 10, 2026, at 17:41, with a reference of #DB10. The total paid is \$5.50, and the status is 'Paid'. The payment method is 'PayPal (Demo)'. There are buttons for 'Download Voucher' and 'Need Help'. Below the booking details is a green box showing the 'Total Spent' as \$5.50, with a note that the user has made 1 booking(s) with Elyora. A tip at the bottom states: 'Tip: All your bookings are saved. You can download vouchers for each booking.'

Cette page récapitule l'historique des réservations effectuées par le touriste. Elle affiche les détails critiques pour chaque transaction :

- Le numéro de réservation et la destination ;
- La date, l'heure et la référence unique ;
- Le statut du paiement (ex: "Payé") et le montant total dépensé.

Une section "Total Spent" calcule automatiquement la somme de toutes les transactions grâce à une fonction **PHP/SQL**. Les utilisateurs peuvent également télécharger leurs reçus. Cette page démontre la robustesse de la base de données et la fiabilité de la gestion des **sessions utilisateur**.

3.1.8 Page de Modification du Profil (Edit Account)



The screenshot displays the 'Edit Your Profile' interface within the Elyora application. The header includes the Elyora logo and navigation links: Home, Profile, Itinerary, Places, My Bookings, Edit Account, and Logout. The form itself is titled 'Edit Your Profile' and contains the following fields:

- Last Name *: sama
- First Name *: Salah
- Email Address *: sailhadudu3@gmail.com
- Telephone *: 0693961260

Below the contact information is a section for 'Change Password (optional)' with three fields:

- Current Password: Enter current password to change
- New Password: Min 8 characters
- Confirm New Password: Repeat new password

At the bottom of the form are two buttons: 'Save Changes' (green) and 'Back to Profile' (grey).

Cette interface permet à l'utilisateur de gérer ses informations personnelles et de sécuriser son compte. Elle offre la possibilité de :

- Mettre à jour ses coordonnées (Nom, Email, Téléphone) ;
- Changer son mot de passe via un formulaire sécurisé.

Le système utilise **PDO** pour mettre à jour les entrées dans la base de données de manière sécurisée. Les changements sont immédiatement pris en compte dans la **session PHP** actuelle. L'ergonomie, travaillée en **CSS**, garantit que l'utilisateur peut modifier ses données rapidement et sans erreur.

3.2 Console Java — Admin - Rapports

La partie Java du projet permet à l'administrateur de gérer les rapports et les statistiques du système en utilisant la programmation orientée objet (OOP).

Les classes `RapportReservation` et `StatistiquesRapport` représentent les données des rapports et peuvent être sauvegardées dans des fichiers grâce à la sérialisation Java, ce qui répond à l'exigence du cours concernant la gestion des fichiers.

La classe `StatistiquesService` contient la logique métier : elle analyse les réservations et calcule différents indicateurs et agrégats statistiques.

L'interface administrateur est organisée avec `MenuAdmin` et `ConsoleHelper`, qui assurent une interaction claire via la console selon une structure proche du modèle MVC, séparant l'affichage, les traitements et les données.

3.2.1 Menu Principal

```
connexion a la base de donnees reussie !
      AGENCE DE VOYAGE - ADMIN
-----
  1. Statistiques & Rapports
  0. Quitter
-----
Votre choix :
```

Cette interface représente le menu principal de l'administrateur dans l'application de gestion d'agence de voyage développée en Java.

Au lancement du programme, le système établit une connexion avec la base de données, puis affiche le message :

« **connexion a la base de donnees reussie !** », indiquant que la communication avec la base de données fonctionne correctement.

Ensuite, le menu « **AGENCE DE VOYAGE - ADMIN** » apparaît pour permettre à l'administrateur d'accéder aux fonctionnalités principales du système.

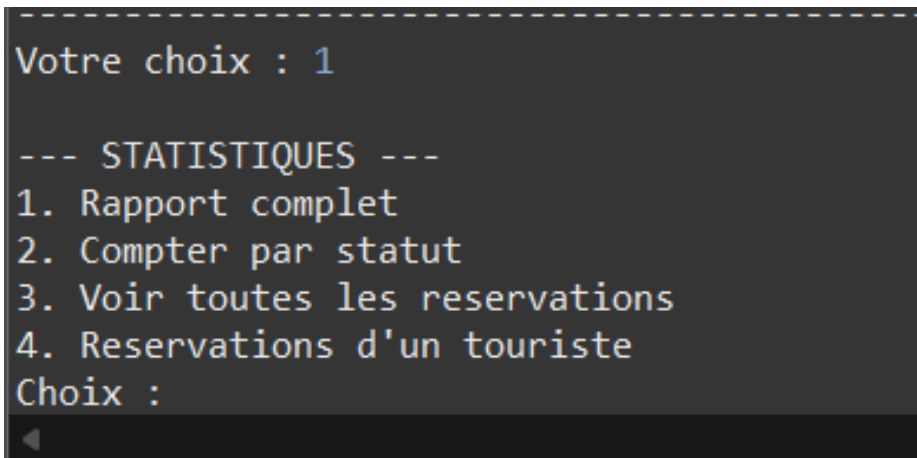
Le menu propose deux options :

- **1. Statistiques & Rapports** : permet d'accéder au module d'analyse des réservations et de génération des rapports statistiques.
- **o. Quitter** : permet de fermer l'application en toute sécurité.

La ligne « **Votre choix :** » attend la saisie de l'utilisateur afin d'exécuter l'option sélectionnée.

Cette interface console est gérée par les classes MenuAdmin et ConsoleHelper, qui assurent une séparation claire entre l'affichage et les traitements selon une architecture inspirée du modèle MVC.

3.2.2 Statistiques & Rapports



```
Votre choix : 1

--- STATISTIQUES ---
1. Rapport complet
2. Compter par statut
3. Voir toutes les reservations
4. Reservations d'un touriste
Choix :
```

Cette interface représente le sous-menu « **STATISTIQUES** » accessible après le choix de l'option numéro 1 dans le menu administrateur.

Ce module permet à l'administrateur d'analyser les réservations enregistrées dans la base de données et de générer différents rapports statistiques.

Les fonctionnalités proposées sont :

- **1. Rapport complet** : génère un rapport global contenant toutes les statistiques importantes du système.
- **2. Compter par statut** : affiche le nombre de réservations selon leur statut (confirmée, annulée, en attente, etc.).
- **3. Voir toutes les réservations** : permet de consulter la liste complète des réservations enregistrées.
- **4. Réservations d'un touriste** : affiche les réservations associées à un touriste spécifique.

Après l’affichage des options, le système attend une saisie utilisateur avec « **Choix :** » afin d’exécuter l’action demandée.

Ce module utilise principalement la classe `StatistiquesService`, responsable du calcul des statistiques et de la génération des rapports à partir des données stockées dans la base de données.

3.2.3 Rapport Complet

```
Choix : 1
===== RAPPORT COMPLET =====
=== RAPPORT STATISTIQUES ===
Total reservations : 11
- Confirmees      : 0
- En attente      : 0
- Annulees        : 0
- Payees          : 11
Total touristes   : 1
Total hotels      : 1
Total restaurants : 1
=====
          AGENCE DE VOYAGE - ADMIN
-----
    1. Statistiques & Rapports
    0. Quitter
-----
Votre choix :
```

Cette interface affiche le rapport complet des statistiques généré par le système après la sélection de l’option « Rapport complet » dans le menu des statistiques.

Le programme analyse automatiquement les données enregistrées dans la base de données afin de produire un résumé global de l’activité de l’agence de voyage.

Les informations affichées sont :

- Total reservations : nombre total de réservations enregistrées.
- Confirmees : nombre de réservations confirmées.
- En attente : nombre de réservations encore en attente de validation.
- Annulees : nombre de réservations annulées.
- Payees : nombre de réservations payées.
- Total touristes : nombre total de touristes enregistrés.
- Total hotels : nombre d’hôtels disponibles.
- Total restaurants : nombre de restaurants enregistrés dans le système.

Ce rapport est généré grâce à la classe `StatistiquesService`, qui calcule les agrégats statistiques à partir des données stockées dans la base de données.

Après l'affichage du rapport, le système retourne automatiquement au menu principal administrateur.

3.2.4 Compter par statut

```
Choix : 2

Entrer le statut (EnAttente / Confirme / Annule / Paye) : Paye
Nombre de reservations 'Paye' : 11
      AGENCE DE VOYAGE - ADMIN

-----
  1. Statistiques & Rapports
  0. Quitter
-----
Votre choix :
```

Cette interface correspond à la fonctionnalité « **Compter par statut** » du module statistiques.

Le système demande à l'administrateur de saisir un statut de réservation parmi les valeurs disponibles :

- EnAttente
- Confirme
- Annule
- Paye

Après la saisie du statut, le programme effectue une recherche dans la base de données et affiche le nombre de réservations correspondant au statut choisi.

Dans cet exemple, le statut saisi est « **Paye** », et le système affiche :
« **Nombre de reservations 'Paye' : 11** ».

Cette fonctionnalité permet à l'administrateur de suivre rapidement l'état des réservations et d'obtenir des statistiques précises sur l'activité du système.

3.2.5 Voir toutes les réservations

```

Choix : 3
--- TOUTES LES RESERVATIONS ---
Reservation #15 | touriste: Mohammed Bouabdellaoui | statut: Paye | du: 2026-05-14 15:40:46 au: 2026-05-15 15:40:46
Reservation #14 | touriste: Mohammed Bouabdellaoui | statut: Paye | du: 2026-05-14 15:40:38 au: 2026-05-15 15:40:38
Reservation #13 | touriste: Mohammed Bouabdellaoui | statut: Paye | du: 2026-05-14 15:40:30 au: 2026-05-15 15:40:30
Reservation #12 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 18:04:21 au: 2026-05-12 18:04:21
Reservation #11 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 18:02:05 au: 2026-05-12 18:02:05
Reservation #10 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 01:10:34 au: 2026-05-12 01:10:34
Reservation #9 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 16:31:21 au: 2026-05-11 16:31:21
Reservation #8 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 16:30:03 au: 2026-05-11 16:30:03
Reservation #7 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 15:59:29 au: 2026-05-11 15:59:29
Reservation #6 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 15:49:26 au: 2026-05-11 15:49:26
Reservation #5 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:58:35 au: 2026-05-11 11:58:35
Reservation #4 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:58:00 au: 2026-05-11 11:58:00
Reservation #3 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:57:41 au: 2026-05-11 11:57:41
Reservation #2 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 00:29:15 au: 2026-05-11 00:29:15
total : 14 element(s)
          AGENCE DE VOYAGE - ADMIN
-----
1. Statistiques & Rapports
0. Quitter
-----
Votre choix :

```

Cette interface représente la fonctionnalité « **Voir toutes les réservations** » du menu statistiques.

Le système récupère toutes les réservations enregistrées dans la base de données et les affiche sous forme de liste détaillée.

Pour chaque réservation, les informations suivantes sont affichées :

- l'identifiant de la réservation ;
- le nom du touriste ;
- le statut de la réservation ;
- la date de début ;
- la date de fin.

Cette fonctionnalité permet à l'administrateur de consulter l'ensemble des réservations afin de suivre l'activité de l'agence de voyage et vérifier les informations enregistrées dans le système.

À la fin de la liste, le programme affiche le nombre total d'éléments récupérés :
« **total : 14 element(s)** ».

Cette opération est réalisée par la classe `StatistiquesService`, qui interroge la base de données et retourne les informations nécessaires pour l'affichage dans l'interface console administrateur.

3.2.6 Voir toutes les réservations

```

Choix : 4
ID du touriste : 1

--- RESERVATIONS DU TOURISTE #1 ---
Reservation #2 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 00:29:15 au: 2026-05-11 00:29:15
Reservation #3 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:57:41 au: 2026-05-11 11:57:41
Reservation #4 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:58:00 au: 2026-05-11 11:58:00
Reservation #5 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 11:58:35 au: 2026-05-11 11:58:35
Reservation #6 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 15:49:26 au: 2026-05-11 15:49:26
Reservation #7 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 15:59:29 au: 2026-05-11 15:59:29
Reservation #8 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 16:30:03 au: 2026-05-11 16:30:03
Reservation #9 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-10 16:31:21 au: 2026-05-11 16:31:21
Reservation #10 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 01:10:34 au: 2026-05-12 01:10:34
Reservation #11 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 18:02:05 au: 2026-05-12 18:02:05
Reservation #12 | touriste: hamza chdaoui | statut: Paye | du: 2026-05-11 18:04:21 au: 2026-05-12 18:04:21
total : 11 element(s)
          AGENCE DE VOYAGE - ADMIN
-----
1. Statistiques & Rapports
0. Quitter
-----
Votre choix :

```

Cette interface représente la fonctionnalité « **Réservations d'un touriste** » du module statistiques et rapports.

Après avoir sélectionné l'option **4**, le système demande à l'administrateur de saisir l'identifiant du touriste avec le message :

« **ID du touriste :** ».

Une fois l'identifiant entré, le programme recherche dans la base de données toutes les réservations associées à ce touriste et les affiche sous forme de liste détaillée.

Pour chaque réservation, le système affiche :

- l'identifiant de la réservation ;
- le nom du touriste ;
- le statut de la réservation ;
- la date de début ;
- la date de fin.

Dans cet exemple, le système affiche toutes les réservations du touriste ayant l'identifiant **1**, avec un total de :

« **total : 11 element(s)** ».

Cette fonctionnalité permet à l'administrateur :

- de consulter l'historique des réservations d'un client ;
- de suivre l'activité d'un touriste spécifique ;
- de vérifier les informations liées aux réservations enregistrées.

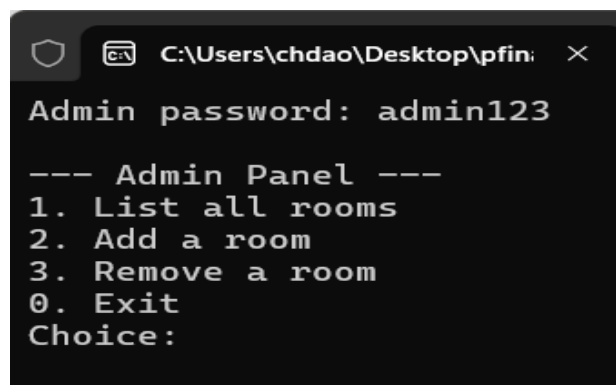
Le traitement est assuré par la classe `StatistiquesService`, qui effectue les requêtes nécessaires pour récupérer les données depuis la base de données et les afficher dans l'interface console administrateur.

3.3 Console C++ — Admin – Operations

Le module Admin utilise C++ pour gérer les données en mémoire à travers des structs comme Chambre, Reservation, MenuItem et Order, ainsi que des classes comme Admin et Database.

L'utilisation des vecteurs et de la gestion directe des données permet de démontrer les bases de la programmation orientée objet (POO) sans utiliser de dépendances externes.

3.3.1 Interface du panneau d'administration



```
C:\Users\chdao\Desktop\pfin: X
Admin password: admin123

--- Admin Panel ---
1. List all rooms
2. Add a room
3. Remove a room
0. Exit
Choice:
```

Cette interface représente le panneau d'administration du système de gestion d'hôtel développé en C++. Après l'authentification de l'administrateur avec un mot de passe (**admin123**), le programme affiche un menu contenant plusieurs options de gestion des chambres.

Les fonctionnalités proposées sont :

1. Lister toutes les chambres disponibles dans le système.
2. Ajouter une nouvelle chambre avec son type et son prix.
3. Supprimer une chambre existante.
4. Quitter le panneau d'administration.

Cette partie du projet démontre :

- L'utilisation des classes comme Admin et Database.
- La gestion des menus avec des structures de contrôle (switch, if, while).
- La manipulation des données en mémoire grâce aux vector.

3.3.2 Affichage de toutes les chambres

```
Choice: 1

--- All Rooms ---
Room      Type      Price/Night  Status
-----
101       Single      $80.00       Available
102       Single      $80.00       Available
201       Double      $130.00      Available
202       Double      $130.00      Available
301       Suite       $250.00      Available
```

Cette interface montre la fonctionnalité “**List all rooms**” du panneau d’administration. Lorsque l’administrateur choisit l’option 1, le système affiche toutes les chambres enregistrées dans la base de données en mémoire.

Les informations affichées pour chaque chambre sont :

- Le numéro de chambre.
- Le type (Single, Double, Suite).
- Le prix par nuit.
- Le statut (Available).

Les données sont stockées dans une structure Room (ou Chambre) puis enregistrées dans un `vector<Room>`. Le programme parcourt ensuite ce vecteur à l’aide d’une boucle pour afficher les informations sous forme de tableau.

Cette fonctionnalité permet de démontrer :

- La manipulation des structures (struct).
- L’utilisation des vecteurs (vector).
- L’affichage formaté des données en console.

3.3.3 Ajout d'une nouvelle chambre

```
Choice: 2
Room number: 1
Type (Single/Double/Suite): Suite
Price per night: $540
[Admin] Room 1 (Suite) added at $540.00/night.
```

Cette interface représente la fonctionnalité “**Add a room**” du module administrateur. Lorsque l’administrateur choisit l’option 2, le système demande :

- Le numéro de la chambre.
- Le type de chambre (Single, Double, Suite).
- Le prix par nuit.

Après la saisie des informations, une nouvelle chambre est créée puis ajoutée au vecteur contenant toutes les chambres du système. Le programme affiche ensuite un message de confirmation indiquant que la chambre a été ajoutée avec succès.

Cette fonctionnalité illustre :

- La création d’objets à partir des structures.
- L’ajout dynamique de données dans un vector.
- L’interaction utilisateur via la console et la validation des entrées.

3.3.4 Suppression d'une chambre existante

```
--- Admin Panel ---  
1. List all rooms  
2. Add a room  
3. Remove a room  
0. Exit  
Choice: 3  
Room number: 101  
[Admin] Room 101 removed.
```

Cette interface illustre la fonctionnalité “**Remove a room**” du panneau d’administration. Lorsqu’un administrateur choisit l’option 3, le système lui demande de saisir le numéro de la chambre qu’il souhaite retirer de la base de données.

Une fois le numéro saisi (dans cet exemple, la chambre 101), le programme recherche l’élément correspondant dans le vecteur de données et procède à sa suppression. Un message de confirmation “[Admin] Room 101 removed” s’affiche pour valider l’opération.

Cette fonctionnalité démontre :

- **La recherche et le filtrage :** l’utilisation d’itérateurs ou de boucles pour localiser un objet spécifique au sein d’un `std::vector`.
- **La gestion dynamique de la mémoire :** la mise à jour des données en temps réel en supprimant des éléments de la mémoire vive.
- **La robustesse du code :** la manipulation des structures de données pour maintenir l’intégrité de la liste après une modification.

CONCLUSION GÉNÉRALE

Le système d'information Elyora représente une réponse ambitieuse et techniquement aboutie aux défis contemporains du tourisme local. En centralisant la gestion des profils utilisateurs, la génération d'itinéraires personnalisés, la gestion des réservations multi-services et le traitement sécurisé des paiements, il offre une plateforme numérique complète, cohérente et scalable au service d'une expérience touristique authentique.

La modélisation UML produite au cours de ce projet — diagrammes de contexte, de cas d'utilisation, de classes, ainsi que les modèles conceptuel et logique de données — valide l'architecture du système en démontrant clairement les interactions entre les acteurs et les flux d'information. Elle garantit la cohérence fonctionnelle et la robustesse technique du projet.

Sur le plan méthodologique, l'adoption d'une approche Agile Scrum structurée en cinq sprints successifs a permis à l'équipe de livrer des modules fonctionnels de manière itérative, d'intégrer les retours des encadrants en temps réel et d'adapter la conception aux contraintes spécifiques du terrain local d'Oujda. Le Cycle en Y a quant à lui assuré une progression parallèle et cohérente des branches fonctionnelle et technique, aboutissant à un système validé et prêt au déploiement.

En définitive, Elyora incarne une vision ambitieuse mais concrète : celle d'un pont numérique entre technologie et tradition, entre le voyageur en quête d'authenticité et l'habitant porteur d'un patrimoine vivant. Ce projet de synthèse constitue, pour l'ensemble de l'équipe, une expérience formatrice exceptionnelle — tant sur les plans techniques, méthodologiques qu'humains — et pose les jalons d'une application à fort potentiel d'impact social et culturel.

Elyora — Un système d'information au service d'un tourisme durable, authentique et profondément ancré dans les richesses culturelles du territoire.

RÉFÉRENCES ET BIBLIOGRAPHIE

Ouvrages et Références Académiques

- [1] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley Professional.
- [2] Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.
- [3] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org.
- [4] Beck, K., et al. (2001). Manifesto for Agile Software Development. Agile Alliance.
- [5] Fowler, M. (2003). Patterns of Enterprise Application Architecture. Addison-Wesley.
- [6] Merise — Méthode d'analyse et de conception des systèmes d'information. Documentation de référence académique.
- [7] MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/>
- [8] UNWTO (2023). Tourism Highlights — World Tourism Organization Annual Report. United Nations.
- [9] Ministère du Tourisme, de l'Artisanat et de l'Économie Sociale et Solidaire du Maroc (2024). Stratégie nationale du tourisme 2023–2026.

Ressources Techniques

- Documentation officielle UML 2.5 — Object Management Group (OMG). <https://www.omg.org/spec/UML/>
- MySQL Workbench Documentation — Oracle Corporation.
- Draw.io / Lucidchart — Outils de modélisation UML et de conception de diagrammes.
- Figma — Plateforme de conception d'interfaces utilisateur (prototypage UI/UX mobile).
- Visual Studio Code avec extensions Java/Spring — Environnement de développement intégré.

Rapport réalisé par l'équipe Elyora — École des Hautes Études d'Ingénierie d'Oujda — 2025–2026